

Ruprecht-Karls-Universität Heidelberg
Institut für Informatik

Bachelor Thesis
Parallel ASP Planning

Name: Levi Maximilian Klein
Student ID: 4283957
Supervisor: Dr. Gnad
Submission date: 17.02.2026

Ich versichere, dass ich diese Bachelor-Arbeit selbstständig verfasst und nur die angegebenen Quellen und Hilfsmittel verwendet habe.

Heidelberg, dd.mm.yyyy

Abstract

Automated Planning is a fundamental concept in Artificial Intelligence that focuses on finding an sequence of actions to reach a desired goal from an initial state. Answer Set Programming (ASP) offers a powerful way to tackle this challenge by modeling dynamic domains through concise declarative rules. However, traditional sequential encodings often suffer from a search depth explosion due to long plan horizons. This thesis investigates the application of parallel planning techniques to overcome this bottleneck. We first establish the theoretical foundations of planning tasks and define the semantics of sequential and parallel plans ($\forall$ -step and $\exists$ -step). Based on the framework proposed by Dimopoulos et al. 2019, specific ASP encodings for these planning strategies are presented. The empirical evaluation demonstrates that parallel encodings significantly outperform the sequential baseline. Specifically, the $\forall$ -step encoding proves to be the most robust, solving a total of 448 out of 996 problem instances from the collection of the IPC benchmark domains, whereas the sequential encoding solves only 202. Additionally, the inclusion of precomputed mutexes improves performance in highly constrained domains like *Agricola*, raising the total coverage to 457 instances while simultaneously reducing runtime for 417 instances. In contrast, neither the backward-propagating heuristic nor the guess-and-check strategy yielded significant performance improvements compared to the standard configuration, with the guess-and-check strategy solving only 366 instances.

Zusammenfassung

Automatische Planung ist ein fundamentales Konzept in künstlicher Intelligenz, das sich mit der Suche nach einer Abfolge von Aktionen beschäftigt, um ein gewünschtes Ziel von einem Anfangszustand aus zu erreichen. Answer Set Programming (ASP) bietet eine leistungsstarke Möglichkeit, diese Herausforderung anzugehen, indem es dynamische Domänen durch knappe deklarative Regeln modelliert. Allerdings leiden traditionelle sequentielle Codierungen oft unter einer Explosion der Suchtiefe aufgrund langer Planhorizonte. Diese Arbeit untersucht die Anwendung von parallelen Planungstechniken, um diese Hürde zu überwinden. Zunächst werden die theoretischen Grundlagen von Planungsaufgaben etabliert und die Semantik von sequentiellen sowie parallelen Plänen ($\forall$ -step und $\exists$ -step) definiert. Aufbauend auf der Arbeit von Dimopoulos et al. 2019, werden ASP-Codierungen für diese Planungsstrategien vorgestellt. Die empirische Evaluation zeigt, dass parallele Codierungen den sequentiellen Ansatz deutlich übertreffen. Insbesondere erweist sich die $\forall$ -step Codierung als am robustesten: Sie löst insgesamt 448 von 996 Probleminstanzen aus einer IPC-Benchmark-Sammlung, während der sequentielle Ansatz lediglich 202 löst. Darüber hinaus verbessert die Einbeziehung vorab berechneter Mutexe die Leistung in stark eingeschränkten Domänen, wie *Agricola*, wodurch die Gesamtzahl der gelösten Instanzen auf 457 steigt und die Laufzeit für 417 Instanzen reduziert wird. Im Gegensatz dazu erzielen weder die rückwärtsgerichtete Heuristik noch die Guess-and-Check-Strategie signifikante Leistungsverbesserungen im Vergleich zur Standardkonfiguration, wobei die Guess-and-Check-Strategie lediglich 366 Instanzen löste.

Contents

List of Figures	1
1 Introduction	3
1.1 Motivation	3
1.2 Goal of the Thesis	3
1.3 Structure of the Thesis	4
2 Theory	5
2.1 Introduction to Planning Task and Parallel Planning	5
2.2 Introduction to ASP	9
2.2.1 Extension for sequential plans	12
2.2.2 Extension for $\forall$ -step plans	13
2.2.3 Extension for $\exists$ -step plans	14
2.2.4 Extension for relaxed $\exists$ -step plans	15
2.3 Proof of Correctness	16
3 Method	24
3.1 Pipeline Overview	24
3.2 Encoding Adaptation	25
3.2.1 Post-Processing and Serialization	26
3.3 Optimizations	26
3.3.1 Mutexes	26
3.3.2 Guess and Check Optimization	27
3.3.3 Heuristic	28
3.4 Experimental Setup	29
3.4.1 Configurations and Encodings	29
3.4.2 Benchmark Selection and Validation Criteria	30
3.4.3 Computational Environment	30
4 Results	31
4.1 Overall Performance	31
4.1.1 Coverage Analysis	31
4.1.2 Runtime Analysis	34

4.2	Impact of Optimizations	35
4.3	Conclusion and Future Work	37
4.3.1	Conclusion	37
4.3.2	Future Work	38
	Bibliography	39

List of Figures

3.1	Pipeline for testing the different planning encodings	24
4.1	Coverage analysis: Number of solved instances per algorithm across benchmark domains. The heatmap indicates the absolute number of solved instances (left) out of the total available instances (right).	32
4.2	Runtime analysis: Average solving time (in seconds) for successfully solved instances. Red cells indicate higher runtimes, while light yellow cells indicate faster performance.	33
4.3	Average runtime for the Guess-and-Check strategy, broken down into the time spent on the initial guess and the subsequent check.	34
4.4	Runtime comparison between different encodings. Each point represents an instance solved by both encodings, with the x-coordinate indicating the runtime of the first encoding and the y-coordinate that of the second. Points below the diagonal indicate better performance of the first encoding.	36
4.5	Runtime comparison of optimization techniques. Top row: The heuristic shows negligible impact (points on diagonal). Bottom row: Mutexes significantly reduce solver time (c), but preprocessing overhead affects total time on easy instances (d).	37

List of Listings

1	ASP fact representation of the example planning task	11
2	Core for sequential and parallel encodings for planning	12
3	Extension of Listing 2 for encoding sequential plans	13
4	Extension of Listing 2 for encoding $\forall$ -step plans	13
5	Extension of Listing 2 for encoding $\exists$ -step plans	14
6	Alternative Extension for $\exists$ -step plans	14
7	Extension of Listing 2 for encoding relaxed $\exists$ -step plans	15
8	Modifications to the core encoding	25
9	Expansion for mutexes	27
10	Encoding for detecting circular invalidation (Checker)	28
11	Backward propagating heuristic	29

1 Introduction

1.1 Motivation

A central objective of Artificial Intelligence is to equip systems with the ability to act autonomously in complex environments. To achieve this, an agent must be able to know the consequences of its actions and construct a sequence of steps that transforms the current state of the world into a desired goal state. This process, known as Automated Planning, is crucial for applications ranging from logistics to autonomous robotics. A key strength of this approach is its domain independence: generic solvers can be applied to diverse fields without modifying their core algorithms, making it highly flexible.

However, this generality comes at a computational cost. Planning problems are inherently difficult, often falling into NP-hard or PSPACE-complete complexity classes. Given these combinatorial challenges, it is a natural step to leverage powerful declarative paradigms. Consequently, Automated Planning was among the first substantial applications of Answer Set Programming (ASP) (Dimopoulos, Nebel, and Koehler 1997); (Lifschitz 2002). ASP is particularly well-suited for modeling dynamic domains due to its expressive action languages and its roots in non-monotonic reasoning.

Despite the expressive power of ASP, a significant gap remains between *modeling* a problem and solving it *effectively*. Traditional ASP encodings often model plans sequentially, executing one action after another. In complex domains, this leads to an explosion of the search depth (plan horizon T), rendering the search for a solution computationally infeasible. In contrast, the field of Satisfiability Testing (SAT) has demonstrated that reasoning about actions can be made significantly more efficient by exploiting parallelism. By allowing multiple non-conflicting actions to occur simultaneously—utilizing so-called parallel step semantics ($\forall$ -step or $\exists$ -step)—the necessary plan horizon can be drastically reduced. This thesis aims to bridge this gap by investigating how these parallel encoding strategies can be applied within ASP to overcome the limitations of sequential planning.

1.2 Goal of the Thesis

Despite the expressive power of ASP, a significant gap remains between modeling a problem and solving it effectively. Traditional ASP encodings often model plans sequentially, executing one action after another. In complex domains, this leads to an explosion of the search depth

(plan horizon T), rendering the search for a solution computationally infeasible. In contrast, the field of Satisfiability Testing (SAT) has demonstrated that *reasoning about actions* can be made significantly more effective by exploiting parallelism (Kautz, McAllester, and Selman 1996). By allowing multiple non-conflicting actions to occur simultaneously, the necessary plan horizon can be drastically reduced. This thesis builds directly upon the framework proposed by Dimopoulos, Gebser, et al. (2019). It aims to revisit these parallel encoding strategies, investigating how they can be effectively applied within a pipeline to overcome the limitations of sequential planning and enhance overall solver performance through optimizations.

1.3 Structure of the Thesis

This thesis is organized as follows:

Chapter 2 establishes the theoretical foundations. We formally define planning tasks and introduce the semantics for sequential and parallel plans. Subsequently, we provide a brief introduction to Answer Set Programming (ASP), present the specific encodings developed by Dimopoulos, Gebser, et al. (2019), and provide formal proofs of their soundness.

Chapter 3 describes the methodological approach and the implementation pipeline. We detail the translation from PDDL to ASP, introduce optimization strategies and describe the setup for the empirical evaluation.

In Chapter 4, we analyze the results of our experiments, discussing the trade-offs between the different encoding strategies.

Finally, we conclude the thesis with a summary of our contributions and an outlook on future work.

2 Theory

2.1 Introduction to Planning Task and Parallel Planning

Before discussing the specific encodings and optimizations for parallel planning from [Dimopoulos, Gebser, et al. \(2019\)](#), we formally define planning problems with multi-valued state variables. This formalism is based on the work of [Helmert \(2006\)](#); [Dimopoulos, Gebser, et al. \(2019\)](#) and serves as the theoretical foundation for the subsequent ASP encodings.

Definition 1 *Multi-valued planning task*. A STRIPS-like multivalued planning task according to [Helmert \(2006\)](#) is given by a 4 tuple $\langle \mathcal{F}, s_0, s_*, \mathcal{O} \rangle$, where:

- $\mathcal{F}$ is a finite set of state variables, also called **fluents**. Each $x \in \mathcal{F}$ is associated with a finite domain $\text{dom}(x)$. In any given state, a fluent x is assigned exactly one value $v \in \text{dom}(x)$.
- s_0 , the **initial state**, is a total function, such that $s_0(x) \in \text{dom}(x)$ for each $x \in \mathcal{F}$.
- s_* , the **goal conditions**, is a partial function, such that $s_*(x) \in \text{dom}(x)$, for each $x \in \text{dom}(s_*)$.
- $\mathcal{O}$ is a finite set of operators, also called **actions**. An action $a \in \mathcal{O}$ is a tuple $\langle a^c, a^e \rangle$, where a^c is the *precondition* and a^e is the *postcondition* of a . Both a^c and a^e are partial functions, such that $a^c(x) \in \text{dom}(x)$ for each $x \in \text{dom}(a^c)$ and $a^e(x) \in \text{dom}(x)$ for each $x \in \text{dom}(a^e)$.

Remark. In the definition from [Helmert \(2006\)](#), the postcondition of actions, called **conditional effect** is a triplet $\langle x, v, \text{cond} \rangle$. Fluent $x \in \text{mathcal{F}}$ is assigned value $v \in \text{dom}(x)$ iff the condition cond holds in the current state. The condition is partial function, such that $\text{cond}(x) \in \text{dom}(x)$ for each $x \in \text{dom}(\text{cond})$. Note that the postcondition a^e in our definition can be seen as a special case of the conditional effect, where the condition is empty.

Example 1. Let us illustrate the definitions with a small example. The goal is to leave the house by opening the door, going outside, and finally closing the door.

- $\mathcal{F} = \{\text{door}, \text{location}\}$ such that $\text{dom}(\text{door}) = \{\text{open}, \text{closed}\}$ and $\text{dom}(\text{location}) = \{\text{inside}, \text{outside}\}$
- $s_0 = \{\text{door}=\text{closed}, \text{location}=\text{inside}\}$
- $s_* = \{\text{door}=\text{closed}, \text{location}=\text{outside}\}$
- $\mathcal{O} = \{\text{open}, \text{close}, \text{leave}\}$, where

$$\begin{aligned}
open &= \langle \{door=closed\}, \{door=open\} \rangle \\
close &= \langle \{door=open\}, \{door=closed\} \rangle \\
leave &= \langle \{location=inside, door=open\}, \{location=outside\} \rangle
\end{aligned}$$

Intuitively, fluents can be understood as the dynamic properties that describe the configuration of the world. In our example, this world is defined by the *location* of the actor and the status of the *door*. States capture a complete snapshot of the world at a given time. Actions define the possible changes of the world, where the precondition establishes the necessary requirements for the action to be applicable and the postcondition specifies the resulting effect. The goal conditionns represent the desired outcome of the planning task, which is typically partial, meaning we only specify the values we explicitly care about, leaving other variables undefined. The initial state represents the complete snapshot of the world at the start. In general, a state defines the system's configuration at a specific point in time. Currently, our system description remains static, restricted to this single initial state. To introduce dynamics, a transition mechanism is needed, which specifies how actions transform one state into another.

Definition 2 Successor state. Let s be a state and $a \in \mathcal{O}$ an action.

The *successor state* $o(a, s)$, obtained by applying action a in state s , is defined if the precondition of a holds, i.e., $a^c(x) = s(x)$ for each $x \in \text{dom}(a^c)$. Then

$$s'(x) = o(a, s)(x) = \begin{cases} a^e(x) & \text{if } x \in \text{dom}(a^e) \\ s(x) & \text{otherwise} \end{cases} \quad \text{for each } x \in \mathcal{F}.$$

Remark. Let $A_n = \langle a_1, \dots, a_n \rangle$ be a sequence of actions with length n . We define the application of A_n to a state s recursevily by

$$o(A_n, s) = o(a_n, o(A_{n-1}, s)), \text{ where } A_{n-1} = \langle a_1, \dots, a_{n-1} \rangle$$

if $o(A_i, s)$ is defined for all $1 \leq i \leq n$. We additionally define $o(\emptyset, s) := s$ for the empty sequence of actions.

Therefore, a state (or more explicitly a reachable state) is any state that can be obtained from the initial state s_0 by applying a sequence of actions. All reachable states and the transitions between them can be represented as a directed graph. Each node corresponds to a distinct reachable state, while the edges represent the possible actions that can be applied to transition from one state to another. With these fundamental building blocks, the necessary framework to describe a solution for a planning task were established.

Definition 3 Sequential plan. A *sequential plan* is a sequence of actions $A_n = \langle a_1, \dots, a_n \rangle$, such that $s' = o(A_n, s_0)$ is defined and $s'(x) = s_*(x)$ for each $x \in \text{dom}(s_*)$.

Example 2. Consider the planning task from the previous example. One possible solution is $A = \langle open, leave, close \rangle$. Since

$$\begin{aligned}
 o(A, s_0) &= o(\langle open, leave, close \rangle, \{door=closed, location=inside\}) \\
 &= o(close, o(\langle open, leave \rangle, \{door=closed, location=inside\})) \\
 &= o(close, o(leave, o(open, \{door=closed, location=inside\}))) \\
 &= o(close, o(leave, \{door=open, location=inside\})) \\
 &= o(close, \{door=open, location=outside\}) \\
 &= \{door=closed, location=outside\} = s_*
 \end{aligned}$$

Graphically, a solution to a planning task equates to finding a path from the source node s_0 to a target node that satisfies the goal condition. While the sequential approach is intuitive, it is often inefficient searching through the whole state space, since the graph grows exponentially with the number of actions. A possible way to reduce the horizon (i.e., the number of time steps needed to satisfy the goal condition) is to allow multiple actions to be executed simultaneously. Since conflict may arise if actions interfere with each other, arbitrary sets of actions cannot simply be applied at once. Consequently, a fundamental requirement for a set of actions is that they must "agree" on their postcondition. This is necessary, since if two actions attempt to assign different values to the same variable and a fluent can only take one value at a time, the resulting state would be undefined. This ambiguity is resolved by introducing the concept of *confluence*.

Definition 4 confluent. Let $A = \{a_1, \dots, a_n\} \subseteq \mathcal{O}$ be a set of actions.

A is *confluent* : $\iff a_i^e(x) = a_j^e(x)$ for all $1 \leq i < j \leq n$ and each $x \in \text{dom}(a_i^e) \cap \text{dom}(a_j^e)$

Remark. A is confluent $\iff B$ is confluent for each $B \subseteq A$

Example 3. In our initial example, the set $\{open, leave\}$ is confluent because the domains of their effects are disjoint. Similarly, $\{close, leave\}$ is confluent.

To illustrate more complex scenarios, we extend our example with two fluents, *light* and *key*, where $\text{dom}(light) = \text{dom}(key) = \{0, 1\}$, and two corresponding actions:

$$\begin{aligned}
 turnOff &= \langle \{location=inside, light=1\}, \{light=0\} \rangle \\
 getKey &= \langle \{location=inside, key=0\}, \{key=1\} \rangle
 \end{aligned}$$

Additionally, we modify the precondition of *open* to require the key ($open^c = \{door=closed, key=1\}$) and update the goal condition to $s_* = \{location=outside, door=closed, light=0\}$.

In this extended setting, the set $\{getKey, open, leave, turnOff\}$ is also confluent, since they target different variables (*key, door, location, light*), so no conflicts in their effects occur.

The concept of confluence guarantees that the simultaneous execution of actions does not lead to a contradictory state. However, ensuring consistent effects is only one part of the problem, since we the preconditions of the actions must also be considered. Unlike in sequential planning, where preconditions are simply checked in the current state, parallel planning allows for different interpretations of precondition dependencies. These interpretations lead to the different parallel representations of sequential plans, which have been investigated in the literature [Dimopoulos, Nebel, and Koehler \(1997\)](#); [Rintanen, Heljanko, and Niemelä \(2006\)](#); [Wehrle and Rintanen \(2007\)](#) and are specifically implemented and tested using Answer Set Programming in [Dimopoulos, Gebser, et al. \(2019\)](#).

Definition 5 *Parallel representations*. Let s be a state and $A = \{a_1, \dots, a_n\}$ a confluent set of actions. Then A is called:

- $\forall$ -step serializable in s $\iff o(\langle a_{\pi(1)}, \dots, a_{\pi(n)} \rangle, s)$ is defined for **all** permutations π
- $\exists$ -step serializable in s $\iff o(\langle a_{\pi(1)}, \dots, a_{\pi(n)} \rangle, s)$ is defined for **some** permutation π and $a^c(x) = s(x)$ for each $a \in A, x \in \text{dom}(a^c)$
- relaxed $\exists$ -step serializable in s $\iff o(\langle a_{\pi(1)}, \dots, a_{\pi(n)} \rangle, s)$ is defined for **some** permutation π

Remark. A is $\forall$ -step serializable $\Rightarrow A$ is $\exists$ -step serializable $\Rightarrow A$ is relaxed $\exists$ -step serializable.

As seen in the remark, these parallel representations form a hierarchy of restrictiveness regarding how the preconditions of actions in a set A are satisfied. The most restrictive representation is the $\forall$ -step serialization, which requires strong independence among the actions. Since the set must be executable in *any* order, no action in the set is permitted to invalidate the preconditions of another and the precondition of all actions in A must hold in the current state s . In contrast, the $\exists$ -step serialization relaxes the independence requirement. While precondition must still be satisfied in s (i.e., actions in A cannot "enable" each other), conflicts are allowed: One action $a \in A$ may invalidate the precondition of another action $a' \in A$, provided that there exists at least *one* valid execution sequence where the conflicting action a occurs after the affected action a' . Finally, the relaxed $\exists$ -step serialization is the least restrictive approach. It abandons the requirement that all preconditions must initially hold in s . Instead, it allows that an action $a \in A$ can establish the preconditions required by another action $a' \in A$, as long as a valid ordering exists in which the enabling action precedes the dependent one. Based on these serialization concepts, we can now define the corresponding parallel plans.

Definition 6 *Parallel Plans*. Let $\mathcal{A}_m = \langle A_1, \dots, A_m \rangle$ be a sequence of confluent sets of actions. $\mathcal{A}_m$ is a $\forall$ -step, $\exists$ -step or relaxed $\exists$ -step plan iff

- $s_m(x) = s_*(x)$ for each $x \in \text{dom}(s_*)$

- A_i is $\forall$ -step, $\exists$ -step or relaxed $\exists$ -step serializable in s_{i-1} for $1 \leq i \leq m$, where

$$s_i(x) = \begin{cases} a^e(x) & \text{if } \exists a \in A_i \text{ s.t. } x \in \text{dom}(a^e) \\ s_{i-1}(x) & \text{otherwise} \end{cases} \quad \text{for each } x \in \mathcal{F}.$$

Example 4. In our modified example we have:

- $\forall$ -step plan: $\langle \{getKey, turnOff\}, \{open\}, \{leave\}, \{close\} \rangle$
- $\exists$ -step plan: $\langle \{getKey, turnOff\}, \{open\}, \{leave, close\} \rangle$
- relaxed $\exists$ -step plan: $\langle \{getKey, turnOff, open, leave\}, \{close\} \rangle$

While the concept of a sequential plan is straightforward, we are effectively dealing with transitions of the form $s \xrightarrow{a} s'$. In parallel planning, however, we generalize this transition to allow sets of actions, i.e., $s \xrightarrow{A} s'$, effectively merging multiple actions together into a single one. Notice how in our example the constraints on parallelism relax with each definition. In the $\exists$ -step plan, the actions *leave* and *close* can be placed in the same step because there exists a valid sequence (*leave* $\rightarrow$ *close*). The relaxed $\exists$ -step plan achieves the highest compression. Here, actions can trigger each other within the same step: *getKey* establishes the precondition for *open*, which in turn enables *leave*. This "chaining" of effects allows us to reduce the first three sets of the $\forall$ -step plan into a single one.

Summary

In this section, the formal framework for multi-valued planning tasks was established. We defined the static components of the world (fluents and states), the dynamic transition logic (applying actions), and the conditions under which sets of actions can be executed in parallel. With these formal definitions in place, the next section will translate this theoretical framework into concrete Answer Set Programming encodings to solve planning tasks efficiently.

2.2 Introduction to ASP

The following section provides a brief overview of Answer Set Programming (ASP). The core concepts and definitions presented here are largely based on the Potassco User Guide (Gebser, Kaminski, Kaufmann, Lindauer, et al. 2019). Answer Set Programming is a declarative problem-solving approach oriented towards difficult (primarily NP-hard) search problems. Unlike imperative programming, where the programmer specifies the control flow to compute a solution, ASP follows a declarative paradigm: the problem is described as a logic program such that its valid models, called *answer sets*, correspond to the solutions of the problem. This methodology allows for a clean separation between the problem description (the logic program) and the solving algorithms.

The theoretical foundation of ASP lies in the stable model semantics of logic programming (Gelfond and Lifschitz 1988). To formally define these programs, we first introduce their basic

building blocks. The most fundamental units are *atoms* of the form $p(t_1, \dots, t_n)$, where $n \in \mathbb{N}_0$, which can either be true or false. These atoms are combined to form *literals*. A literal is defined as either an atom A or its default negation *not* A . Here, *not* denotes negation as failure, meaning that *not* A is considered true if no proof for A can be derived from the program.

Grounding and Solvers

Atoms often contain variables (e.g., $p(X)$ instead of $p(1)$) to allow for compact problem encodings. However, standard ASP solvers operate on propositional logic and cannot handle variables directly. To bridge this gap, a process called *grounding* is performed before the actual search. During grounding, the symbolic program is transformed into a variable-free program by replacing all variables with every possible combination of domain constants. Consequently, modern ASP systems like *clingo*¹ consist of two main components: the grounder *gringo*, which handles this transformation, and the solver *clasp*, which performs the search for answer sets on the grounded program.

Logic Programs

Based on these definitions, we can specify the structure of the programs used to model planning tasks. A *normal logic program* is a finite set of fact, rules and integrity constraints of the form:

$$A_0 :- L_1, \dots, L_n.$$

The *head* of the rule is the atom A_0 , and the *body* is a conjunction of literals $L_1, \dots, L_n$, where $n \in \mathbb{N}_0$. The symbol ":-" (meaning $\leftarrow$) denotes logical implication: if all literals in the body are true, then the head will be true. We distinguish three specific types of rules based on this structure:

- **Facts** ($n = 0$): Rules with an empty body, implying that the head atom A_0 is unconditionally true.
- **Rules** ($n \geq 1$): Rules that derive the head A_0 only if the body is satisfied.
- **Integrity Constraints** (A_0 is empty): Rules with an empty head, serving as a filter for solution candidates. If the body is satisfied, the rule yields a contradiction. Thus, at least one literal in the body must be false.

Semantics and Planning Encoding

A set of atoms is called a *model* of a logic program if it satisfies all rules, facts, and integrity constraints. In ASP, a model is specifically called an *answer set* (or stable model) if every atom in the model has a non-cyclic derivation.

¹<https://potassco.org/clingo/>


```

1  fluent(door; loc; light; key).
2  value(door, closed). value(door, open).
3  value(loc, in).      value(loc, out).
4  value(light, 0).     value(light, 1).
5  value(key, 0).       value(key, 1).
6
7  init(door, closed). init(loc, in).  init(light, 1). init(key, 0).
8  goal(door, closed). goal(loc, out). goal(light, 0).
9
10 action(open; close; leave; turnOff; getKey).
11 prec(open, door, closed). prec(open, key, 1).      post(open, door, open).
12 prec(close, door, open).      post(close, door, closed).
13 prec(leave, loc, in).      prec(leave, door, open). post(leave, loc, out).
14 prec(turnOff, loc, in).     prec(turnOff, light, 1). post(turnOff, light, 0).
15 prec(getKey, loc, in).      prec(getKey, key, 0).    post(getKey, key, 1).

```

Listing 1: ASP fact representation of the example planning task

We can translate the formal definition of a planning task into such an ASP logic program. Following the example in [Dimopoulos, Gebser, et al. \(2019\)](#), the problem instance is represented purely by facts: fluents are defined via `fluent/1` and their domain via `value/2`, the initial state s_0 is defined via the predicate `init/2`, the goal s_* via `goal/2`, and the pre- and postconditions of actions via `prec/3` and `post/3` respectively. Listing 1 shows the complete fact representation of our example.

Combining these facts with the right encodings, stable models correspond to the sequential, $\forall$ -step, $\exists$ -step or relaxed $\exists$ -step plans.

All planning algorithms presented in this work share a common incremental encoding (Listing 2) using the `incmode` feature of *clingo* ([Gebser, Kaminski, Kaufmann, and Schaub 2014](#)). The program is divided into three subprograms: *base*, *step(t)* and *check(t)*.

The *base* subprogram (Line 2) defines the initial state s_0 , grounding the fluent values at timestep 0. The predicate `holds(X,V,t)` is used to represent fluents $X \in \mathcal{F}$ that have value $V \in \text{dom}(X)$ at timestep t .

The *check(t)* subprogram (Line 4) is parametrized and starts at $t = 0$. The integrity constraint enforces the goal condition, i.e. $s_*(X) = s_t(X)$ for each $X \in \text{dom}(s_*)$. The external atom *query(t)* ensures this check is only active for the current search horizon t (i.e., the current candidate plan length).

The *step(t)* subprogram (Line 6-9) is grounded for each timestep t starting at $t = 1$. The

```

1  #include <incmode>.
2  holds(X, V, 0) :- init(X, V).
3  #program check(t).
4  :- query(t), goal(X, V), not holds(X, V, t).
5  #program step(t).
6  {holds(X, V, t) : value(X, V)} = 1 :- fluent(X).
7  {occurs(A, t)} :- action(A).
8  :- occurs(A, t), post(A, X, V), not holds(X, V, t).
9  :- holds(X, V, t), not holds(X, V, t-1), not occurs(A, t) : post(A, X, V).

```

Listing 2: Core for sequential and parallel encodings for planning

choice rule in Line 6 creates the state candidates for timestep t , assigning every fluent X exactly one value V from its domain and the choice rule in Line 7 generates arbitrary sets of actions $A \in \mathcal{O}$ for the current timestep t in the form of `occurs(A,t)`. These actions and the generated state must obey the integrity constraints in Lines 8 and 9. Line 8 ensures that the postconditions of an occurring action is actually applied in the state. This also asserts the confluence of the set of actions: if the postconditions of two actions a and a' have different values for the same fluent (i.e. $V = a^e(X) \neq a'^e(X) = V'$), the uniqueness of the value in the choice rule (Line 6) would be violated. Finally, Line 9 guarantees that fluents not affected by any action keep their value from the previous timestep. It rejects any change in a fluent's value that is not explained by an action.

In summary, Listing 2 establishes the state transitions with the postcondition of the chosen actions. However, it does not yet verify the *applicability* of these actions regarding their preconditions, nor does it enforce the specific serializability requirements (e.g. acyclicity) of the parallel semantics.

2.2.1 Extension for sequential plans

To restrict the generic transition system to valid *sequential plans*, we need to ensure that preconditions are satisfied and that each execution is restricted to a single action at a time. This is achieved by the extension shown in Listing 3.

```

1 :- occurs(A, t), prec(A, X, V), not holds(X, V, t-1).
2 :- #count{A : occurs(A, t)} > 1.

```

Listing 3: Extension of Listing 2 for encoding sequential plans

Line 2 implements the applicability check, enforcing that for any action occurring at timestep t , all its preconditions must hold in the preceding state s_{t-1} . To ensure the plan remains sequential, Line 3 guarantees that only one action can occur in each timestep t .

2.2.2 Extension for $\forall$ -step plans

```

1 :- occurs(A, t), prec(A, X, V), not holds(X, V, t-1).
2 :- occurs(A, t), prec(A, X, V), not post(A, X, _), not holds(X, V, t).
3 single(X, t) :- occurs(A, t), prec(A, X, V1), post(A, X, V2), V1 != V2.
4 :- single(X, t), #count{A : occurs(A, t), post(A, X, V)} > 1.

```

Listing 4: Extension of Listing 2 for encoding $\forall$ -step plans

The extension for $\forall$ -step plans is shown in Listing 4. Line 1 is identical to the sequential encoding, ensuring that the preconditions of all chosen actions are satisfied in the previous state s_{t-1} . To create a valid $\forall$ -step plan, we must ensure that each set of actions is executable in *any* order. Since Line 8 of the common core (Listing 2) already enforces confluence, the remaining requirement is that no action is allowed to invalidate the preconditions of another action. We distinguish two types of interactions between an action a and a fluent X in its precondition: Action a has X in its preconditions but does not change it, (i.e. $X \in \text{dom}(a^c) \cap \text{dom}(a^e)$ and $a^c(X) = a^e(X)$) or $X \in \text{dom}(a^c)$ and $X \notin \text{dom}(a^e)$) or action a has X in its preconditions and also changes its value (i.e. $X \in \text{dom}(a^e)$ and $a^c(X) \neq a^e(X)$). Line 2 handles the first case by ensuring that X has to remained unchanged (" $_$ " is a wildcard for any value). However, the second case requires strict isolation of these kinds of actions. If an action a invalidates its own precondition on fluent X , no other action a' in the same timestep is allowed to modify X . If such an action a' existed, the execution order $\langle a', a \rangle$ would fail because a' would change X , invalidating the precondition of a . To enforce this, Line 3 marks such actions using the predicate `single(X,t)`. The integrity constraint in Line 4 then ensures that for any fluent X , at most one such action occurs per time step, preventing any interference.

```

1 :- occurs(A, t) , prec(A, X, V), not holds(X, V, t-1).
2 apply(A1, t) :- action(A1),
3     ready(A2, t) : post(A1, X, V1), prec(A2, X, V2), A1 != A2, V1 != V2.
4 ready(A, t) :- action(A), not occurs(A, t).
5 ready(A, t) :- apply(A, t).
6 :- action(A) , not ready(A, t).

```

Listing 5: Extension of Listing 2 for encoding $\exists$ -step plans

```

1 :- occurs(A, t), prec(A, X, V), not holds(X, V, t-1).
2 #edge((A1, t), (A2, t)): occurs(A1, t),
3     post(A1, X, V1), prec(A2, X, V2), A1 != A2, V1 != V2.

```

Listing 6: Alternative Extension for $\exists$ -step plans

2.2.3 Extension for $\exists$ -step plans

Dimopoulos, Gebser, et al. (2019) presents two different approaches to encode the extension for $\exists$ -step plans, shown in Listings 5 and 6. As before, Line 1 in both listings enforces the standard precondition check. The difference lies in how they enforce the serializability requirement.

Listing 6 implements a graph-based approach. Since the common core (Listing 2) already establishes confluence, the only remaining requirement is the existence of a valid execution order. This effectively means excluding circular interference among the chosen actions.

This lends itself to an implementation using the `#edge` directive (Gebser, Kaminski, Kaufmann, Ostrowski, et al. 2016) of *clingo*, where built-in acyclicity checking (Bomanson et al. 2016) is utilized.

The underlying structure corresponds to the *conflict graph*. Following the definition in Dimopoulos, Gebser, et al. (2019), we define the graph over all possible actions, with edges originating from the currently active set.

Definition 7 Conflict Graph. Let A_t be the set of actions occurring at time t . The conflict graph $G_t = (\mathcal{O}, E)$ for timestep t is a directed graph where

$$E = \{\langle a, a' \rangle \mid \{a, a'\} \in A_t \times \mathcal{O} \setminus \{a\}, \exists X \in \text{dom}(a^e) \cap \text{dom}(a'^c) : a^e(X) \neq a'^c(X)\}$$

Intuitively, there is a directed edge from a to a' , if action a invalidates a precondition of action a' . Line 2 of Listing 6 directly translates these edges into the `#edge` syntax. A valid

serialization of the actions corresponds to a reversed topological ordering of the subgraph induced by A_t . Consequently, any answer set containing a cycle within the set of occurring actions is automatically rejected by the solver's cycle-checking logic.

Listing 5, on the other hand, implements this acyclicity checking by a recursive derivation of the predicates `apply` and `ready`. Line 4 defines the base case: Any action that does not occur is considered *ready* (since it does not interfere with the execution order). Line 5 marks occurring actions: If an action is successfully applied, it is also considered *ready*. Line 2-3 implements the ordering logic. An action a can only be applied (`apply`) if all other actions a' whose preconditions it invalidates are already applied or do not occur (`ready`). Line 6 is the final constraint: It requires that every occurring action must eventually become *ready*. If the subgraph induced by A_t contains a cycle (e.g., a invalidates a' and a' invalidates a), neither `apply(a,t)` nor `apply(a',t)` can ever be derived, because each would wait for the other to be ready first. Consequently, the constraint in Line 6 would be violated, causing the solver to reject the model.

2.2.4 Extension for relaxed $\exists$ -step plans

```

1 reach(X, V, t) :- holds(X, V, t-1).
2 reach(X, V, t) :- occurs(A, t), apply(A, t), post(A, X, V).
3 apply(A1, t) :- action(A1), reach(X, V, t) : prec(A1, X, V);
4   ready(A2, t) : post(A1, X, V1), prec(A2, X, V2), A1 != A2, V1 != V2.
5 ready(A, t) :- action(A), not occurs(A, t).
6 ready(A, t) :- apply(A, t).
7 :- action(A), not ready(A, t).

```

Listing 7: Extension of Listing 2 for encoding relaxed $\exists$ -step plans

Finally, we present the extension from Dimopoulos, Gebser, et al. (2019) for relaxed $\exists$ -step plans in Listing 7. Structurally, it shares the same derivation approach with the standard $\exists$ -step encoding, but it relaxes the applicability check. The other encodings required that preconditions strictly had to hold in the previous state s_{t-1} . Here, actions within the same parallel set can provide "internal support" for each other. This means an action is applicable if its preconditions are established either by the previous state or by another action occurring earlier in the valid execution sequence of the current step.

This logic is implemented by the predicate `reach/3`, which represents the values of fluents that are currently available. A value is reachable if it holds in the previous state (Line 1) or if it is obtained by an action that has already been applied in the current internal sequence (Line

2). The rule in Line 3 and 4 now checks against the reach predicate instead of the static state. Therefore, an action can safely be applied if its preconditions are reachable. As in the standard $\exists$ -step encoding, we must still check acyclicity. Consequently, the integrity constraint in Line 7 ensures that no such circular dependencies exist, guaranteeing a valid serialization.

2.3 Proof of Correctness

Having presented the ASP encodings for sequential and the different parallel semantics, we now present the proof from [Dimopoulos, Gebser, et al. \(2019\)](#) that formally verify their correctness. The goal is to establish a one-to-one correspondence between the stable models of our logic programs and the valid plans as defined in Definition 6. We adopt the notation from [Dimopoulos, Gebser, et al. \(2019\)](#) and decompose the logic programs into their constituent parts:

- I denotes the set of facts representing the specific planning task instance $\langle \mathcal{F}, s_0, s_*, \mathcal{O} \rangle$ (as in Definition 1).
- B corresponds to the rule in the **base** subprogram (Listing 2, Line 2).
- $Q(t)$ represents the goal verification rule defined in the **check(t)** subprogram (Listing 2, Line 4).
- $S(t)$ denotes the transition rules defined in the **step(t)** subprogram (Listing 2, Lines 6-9).
- $S^p(t)$ is the specific extension for $S(t)$, where:
 - $p = s$ for the sequential extension (Listing 3),
 - $p = \forall$ for the $\forall$ -step extension (Listing 4),
 - $p = \exists$ for the $\exists$ -step extension (Listing 5),
 - $p = E$ for the alternative $\exists$ -step extension (Listing 6),
 - $p = R$ for the relaxed $\exists$ -step extension (Listing 7).

Using these components, the complete logic program for a horizon n is defined as:

$$P^p(n) := I \cup B \cup \bigcup_{t=1}^n (S(t) \cup S^p(t) \cup Q(t)) \cup \{\text{query}(n).\}$$

Theorem 1 (Correctness of sequential plans ([Dimopoulos, Gebser, et al. 2019](#))). Let $A_n := \langle a_1, \dots, a_n \rangle$, $a_t \in \mathcal{O}$ for each $1 \leq t \leq n$, be a sequence of actions. Then,

$$A_n \text{ is a sequential plan} \iff P^s(n) \text{ has a stable model } M \text{ such that} \\ \{\langle a, t \rangle \mid \text{occurs}(a, t) \in M\} = \{\langle a_t, t \rangle \mid 1 \leq t \leq n\}.$$

Proof. We prove both directions separately.

($\Rightarrow$): Assume A_n is a sequential plan.

We construct a candidate set M based on the plan execution and show it is indeed a stable model of $P^s(n)$. The rule in B ensures that $\text{holds}(\mathbf{X}, \mathbf{V}, 0) = s_0(X)$ for each $X \in \mathcal{F}$. The choice rule (Line 5 of Listing 3) enforces that for any $1 \leq t \leq n$ and $X \in \mathcal{F}$, exactly one value V is assigned. Moreover, the integrity constraints (Lines 8 and 9) in $S(t)$ dictate the transition logic between states. For any $1 \leq t \leq n$, the value V of a fluent X is determined by:

$$\text{holds}(\mathbf{X}, \mathbf{V}, \mathbf{t}) \leftrightarrow \begin{cases} V = a_t^e(X) & \text{if } X \in \text{dom}(a_t^e) \\ V = s_{t-1}(X) & \text{otherwise} \end{cases}$$

Therefore, the set of **holds**-atoms in any stable model must satisfy:

$$\{\langle X, V, t \rangle \mid \text{holds}(\mathbf{X}, \mathbf{V}, \mathbf{t}) \in M\} = \{\langle X, V, t \rangle \mid X \in \mathcal{F}, V = o(A_t, s_0)(X), 0 \leq t \leq n\} \quad (1)$$

Since the action sequence is fixed by the plan assumption ($\{\langle a, t \rangle \mid \text{occurs}(\mathbf{a}, \mathbf{t}) \in M\} = \{\langle a_t, t \rangle \mid 1 \leq t \leq n\}$), the only candidate for a stable model $P^s(n)$ is:

$$\begin{aligned} M = & I \cup \{\text{occurs}(a_t, \mathbf{t}) \mid 1 \leq t \leq n\} \\ & \cup \{\text{holds}(\mathbf{X}, \mathbf{V}, \mathbf{t}) \mid (X, V, t) \text{ satisfies (1)}\} \cup \{\text{query}(\mathbf{n})\}. \end{aligned}$$

By the definition of a sequential plan, $o(a_t, s_{t-1})$ is defined for all $1 \leq t \leq n$, and the goal condition $s_n(X) = s_*$ for each $X \in \text{dom}(s_*)$ is met. Thus, the integrity constraint regarding preconditions (Line 1, Listing 3) and the goal check (Line 4, Listing 2) are satisfied. Since M is a set satisfying all rules and constraints, it is indeed a stable model of $P^s(n)$.

($\Leftarrow$): Assume that M is a stable model of $P^s(n)$ s.t. $\{\langle a, t \rangle \mid \text{occurs}(\mathbf{a}, \mathbf{t}) \in M\} = \{\langle a_t, t \rangle \mid 1 \leq t \leq n\}$.

We prove by induction over t , that the following equation holds for each $1 \leq t \leq n$:

$$\{\langle X, V, t \rangle \mid \text{holds}(\mathbf{X}, \mathbf{V}, \mathbf{t}) \in M\} = \{\langle X, V, t \rangle \mid X \in \mathcal{F}, V = o(A_t, s_0)(X), 0 \leq t \leq n\} \quad (2)$$

Base case $t = 0$:

The rule in B directly implies that $\{\langle X, V, \rangle \mid \text{holds}(\mathbf{X}, \mathbf{V}, 0) \in M\} = \{\langle X, V \rangle \mid X \in \mathcal{F}, V = s_0(X)\}$, satisfying the equation for $t = 0$.

Induction step $t - 1 \mapsto t$:

Assume (2) holds for $t - 1$, i.e., the state at $t - 1$ corresponds to $s_{t-1} = o(A_{t-1}, s_0)$. The integrity constraint in Line 1 of Listing 3 in $S^s(t)$ ensures that the successor state $s_t = o(a_t, o(A_{t-1}, s_0))$ is defined and the choice rule in Line 6 of Listing 2, together with

the integrity constraints in $S(t)$ guarantee that

$$\{\langle X, V, t \rangle \mid \text{holds}(\mathbf{X}, \mathbf{V}, \mathbf{t}) \in M\} = \{\langle X, V, t \rangle \mid X \in \mathcal{F}, V = o(A_t, s_0)(X), 0 \leq t \leq n\}.$$

Thus, (2) holds for each $1 \leq t \leq n$. Given $\text{query}(\mathbf{n})$, the integrity constraint in $Q(n)$ yields that $o(A_n, s_0) = s_*(X)$ for each $X \in \text{dom}(s_*)$ and therefore A_n is a sequential plan. $\square$

Theorem 2 (Correctness of parallel plans (Dimopoulos, Gebser, et al. 2019)). Let $\mathcal{A} = \langle A_1, \dots, A_m \rangle$ be a sequence of sets of actions, $A_t \subseteq \mathcal{O}$ for $1 \leq t \leq m$ and $p \in \{\forall, \exists, R\}$. Then,

$$\begin{aligned} \mathcal{A} \text{ is a } p\text{-step plan} &\iff P^p(m) \text{ has a stable model } M \text{ such that} \\ &\quad \{\langle a, t \rangle \mid \text{occurs}(a, t) \in M\} = \{\langle a, t \rangle \mid a \in A_t, 1 \leq t \leq m\} \end{aligned}$$

Proof. We again prove both directions separately.

($\Rightarrow$): Assume $\mathcal{A} = \langle A_1, \dots, A_m \rangle$ is a $\forall$ -/ $\exists$ -/relaxed $\exists$ -step plan.

For each $1 \leq t \leq m$, let s_t be the state defined by applying the set A_t to s_{t-1} . Due to the confluence of A_t , s_t is well-defined as

$$s_t(X) = \begin{cases} a^e(X) & \text{for each } a \in A_t \text{ where } X \in \text{dom}(a^e) \\ s_{t-1}(X) & \text{otherwise} \end{cases} \quad \text{for each } X \in \mathcal{F}.$$

Analogous to the proof of Theorem 1, consider any stable model M of $P^p(m)$ such that the action occurrences match the plan, i.e. $\{\langle a, t \rangle \mid \text{occurs}(\mathbf{a}, \mathbf{t}) \in M\} = \{\langle a, t \rangle \mid a \in A_t, 1 \leq t \leq m\}$. The rule in B , together with the choice rule and integrity constraints in $S(t)$, dictates the values of all fluents based on these actions. Thus, the set of **holds**-atoms in M is determined as:

$$\{\langle X, V, t \rangle \mid \text{holds}(\mathbf{X}, \mathbf{V}, \mathbf{t}) \in M\} = \{\langle X, V, t \rangle \mid X \in \mathcal{F}, V = s_t(X), 0 \leq t \leq m\}.$$

We now distinguish the different cases of parallel plans:

Case 1: $\mathcal{A}$ is a $\forall$ -step plan.

For $1 \leq t \leq m$, the atom $\text{single}(\mathbf{X}, \mathbf{t})$ defined in $S^\forall(t)$ (Line 2 of Listing 4) is derived iff there exists some action $a \in A_t$ that invalidates its own precondition, i.e., $X \in \text{dom}(a^c) \cap \text{dom}(a^e)$ and $a^c(X) \neq a^e(X)$. Hence, $P^\forall(m)$ cannot have any stable model M

apart from

$$\begin{aligned}
M = & I \cup \{\text{occurs}(\mathbf{a}, \mathbf{t}) \mid a \in A_t, 1 \leq t \leq m\} \\
& \cup \{\text{holds}(\mathbf{X}, \mathbf{V}, \mathbf{t}) \mid X \in \mathcal{F}, V = s_t(X), 0 \leq t \leq m\} \\
& \cup \{\text{single}(\mathbf{X}, \mathbf{t}) \mid a \in A_t, X \in \text{dom}(a^c) \cap \text{dom}(a^e), a^c(X) \neq a^e(X), 1 \leq t \leq m\} \\
& \cup \{\text{query}(\mathbf{m})\}
\end{aligned}$$

such that $\{\langle a, t \rangle \mid \text{occurs}(\mathbf{a}, \mathbf{t}) \in M\} = \{\langle a, t \rangle \mid a \in A_t, 1 \leq t \leq m\}$.

It remains to show that M satisfies the integrity constraints in $S^\forall(t)$. Since for any $1 \leq t \leq m$, the successor state $s_t = o(\langle a_1, \dots, a_k \rangle, s_{t-1})$ is defined for *any* sequence $\langle a_1, \dots, a_k \rangle$ with $\{a_1, \dots, a_k\} = A_t$, no action can interfere with the precondition of another action. Consequently, if $\text{single}(\mathbf{X}, \mathbf{t})$ is derived (meaning an action invalidates its own precondition on X), no other action can modify X , as this would create a conflict in at least one permutation. Therefore, the integrity constraint restricting the count of actions affecting X with $\text{single}(\mathbf{X}, \mathbf{t})$ is satisfied. Analogously, the constraint requiring preconditions to remain stable (Line 1 of Listing 4) is satisfied since $s_{t-1}(X) = a^c(X)$ for each $a \in A_t$, $X \in \text{dom}(a^c)$ and $1 \leq t \leq m$. One can check that the remaining rules and integrity constraints in $S(t)$ are satisfied as well, so M is indeed a stable model of $P^\forall(m)$.

Case 2: $\mathcal{A}$ is a $\exists$ -step plan.

We analyze the programs $P^\exists(m)$ and $P^E(m)$ separately, starting with the acyclicity-based program $P^E(m)$.

Analysis of $P^E(m)$:

By definition of an $\exists$ -step plan, for every timestep $1 \leq t \leq m$, there is a sequence of actions $\langle a_1, \dots, a_k \rangle$ with $\{a_1, \dots, a_k\} = A_t$ such that $o(\langle a_1, \dots, a_k \rangle, s_{t-1}) = s_t$ is defined. A valid serialization requires that no action invalidates the precondition of a subsequent action. Formally, let $G_t = (A_t, E_t)$ be the conflict graph for timestep t , where an edge represents a destroyed precondition:

$$E_t = \{(a, a') \in A_t \times A_t \mid a \neq a', \exists X \in \text{dom}(a^e) \cap \text{dom}(a'^c) : a^e(X) \neq a'^c(X)\}.$$

Since a valid serialization exists, $\langle a_1, \dots, a_k \rangle$ constitutes a reversed topological ordering of G_t , i.e. there exists a sequence of applied actions. A directed graph possesses a topological ordering iff it is a Directed Acyclic Graph (DAG). Thus, G_t must be acyclic.

Now consider the candidate model M :

$$\begin{aligned}
M = & I \cup \{\text{occurs}(\mathbf{a}, \mathbf{t}) \mid a \in A_t, 1 \leq t \leq m\} \\
& \cup \{\text{holds}(\mathbf{X}, \mathbf{V}, \mathbf{t}) \mid X \in \mathcal{F}, V = s_t(X), 0 \leq t \leq m\} \cup \{\text{query}(\mathbf{m})\}.
\end{aligned}$$

The `#edge` directive in Listing 6 (Lines 2-3) generates an edge between two occurring actions (a, t) and (a', t) exactly if the condition for E_t is met. Consequently, the set of edges declared in the ASP model corresponds exactly to the union of all conflict graphs $\bigcup_{t=1}^m E_t$. Since each G_t is acyclic and the graphs for different timesteps $1 \leq t < t' \leq m$ are vertex-disjoint $((a, t), \neq (a, t'))$, the overall graph induced by M is acyclic. Therefore, the acyclicity constraint is satisfied, and M is a stable model of $P^E(m)$.

Analysis of $P^\exists(m)$:

The rules defining `apply` and `ready` in $S^\exists(t)$ (Lines 2-5 in Listing 5) effectively traverse the dependency graph G_t . Since G_t is acyclic, there exists a reversed topological ordering of the actions $a \in A_t$. The derivation of atoms proceeds along this ordering: Actions with no dependencies (sinks in G_t) are derived as `apply` first, making them `ready`. This, in turn, satisfies the conditions for their predecessors. Because G_t is finite and acyclic, this inductive process reaches a fixpoint where `apply(a, t)` and `ready(a, t)` hold for each $a \in \mathcal{O}$ and $1 \leq t \leq m$. Consequently, the integrity constraint requiring all actions to be ready (Line 6 in Listing 5) is satisfied. Thus, $P^\exists(m)$ extends the base model M to:

$$M' = M \cup \{\text{apply}(a, t) \mid a \in \mathcal{O}, 1 \leq t \leq m\} \cup \{\text{ready}(a, t) \mid a \in \mathcal{O}, 1 \leq t \leq m\}$$

such that $\{\langle a, t \rangle \mid \text{occurs}(a, t) \in M'\} = \{\langle a, t \rangle \mid a \in A_t, 1 \leq t \leq m\}$. As established previously, the state transition rules are satisfied by M (and thus by M'). Since the additional constraints in $P^\exists(m)$ regarding circularity are also satisfied by the completeness of the `ready` derivation, M' is indeed a stable model of $P^\exists(m)$.

Case 3: $\mathcal{A}$ is a relaxed $\exists$ -step plan.

For each $1 \leq t \leq m$, by definition of a relaxed $\exists$ -step plan, there exists a serialization $\langle a_1, \dots, a_k \rangle$ of A_t such that $\{a_1, \dots, a_k\} = A_t$ and the sequential execution is valid. This means $o(a_i, o(\langle a_1, \dots, a_{i-1} \rangle, s_{t-1}))$ is defined for all $1 \leq i \leq k$.

Consequently, the preconditions of any action a_i are satisfied by the state resulting from the application of its predecessors in $\langle a_1, \dots, a_k \rangle$. The rules in $S^R(t)$ (Listing 7) simulate this reachability:

- (i) The atom `reach(X, V, t)` is derived for all values V that are either present in s_{t-1} or produced by an applied action.
- (ii) The atom `apply(a, t)` is derived for any action $a \in \mathcal{O}$ whose preconditions are satisfied by the reachable values.
- (iii) The atom `ready(a, t)` is derived for all $a \in \mathcal{O}$ (either because they are applied or because they do not occur).

Since the serialization $\langle a_1, \dots, a_k \rangle$ is valid, the least fixpoint of these rules guarantees that `apply(a, t)` is derived for each $a \in A_t$.

Thus, the stable model M of $P^R(m)$ corresponds to

$$\begin{aligned}
M = & I \cup \{\text{occurs}(a, t) \mid a \in A_t, 1 \leq t \leq m\} \\
& \cup \{\text{holds}(X, V, t) \mid X \in \mathcal{F}, V = s_t(X), 0 \leq t \leq m\} \\
& \cup \{\text{reach}(X, V, t) \mid X \in \mathcal{F}, V \in \{s_{t-1}, s_t\}, 1 \leq t \leq m\} \\
& \cup \{\text{apply}(a, t) \mid a \in \mathcal{O} : a^c(X) \in \{s_{t-1}(X), s_t(X)\} \text{ for each } X \in \text{dom}(a^c)\} \\
& \cup \{\text{ready}(a, t) \mid a \in \mathcal{O}, 1 \leq t \leq m\} \\
& \cup \{\text{query}(m)\}
\end{aligned}$$

Since M contains $\text{ready}(a, t)$ for all actions $a \in \mathcal{O}$, the integrity constraint in Line 7 of Listing 7 is satisfied. Furthermore, all state transition constraints are met by the definition of s_t . Therefore, M is a stable model of $P^R(m)$.

($\Leftarrow$): Assume that M is a stable model of $P^p(m)$ such that $\{\langle a, t \rangle \mid \text{occurs}(a, t) \in M\} = \{\langle a, t \rangle \mid a \in A_t, 1 \leq t \leq m\}$.

For any $1 \leq t \leq m$, the choice rule in Line 6 of Listing 2 ensures that the set of **holds**-atoms induces a state function s_t defined by $s_t(X) = V \Leftrightarrow \text{holds}(X, V, t) \in M$.

Moreover, the rules and integrity constraints in $S(t)$ dictate the state transition. Specifically, the integrity constraint in Line 8 of Listing 2 ensure that for each $a \in A_t$ and $X \in \text{dom}(a^e)$, $s_t(X) = a^e(X)$ holds. Since M is a stable model, contradicting effects on the same variable are impossible (as they would violate the constraints), which implies that A_t is confluent. For unaffected fluents, the integrity constraint in Line 9 of Listing 2 yield $s_t(X) = s_{t-1}(X)$ for each $X \in \mathcal{F} \setminus \bigcup_{a \in A_t} \text{dom}(a^e)$.

Finally, given the fact $\text{query}(m)$, the integrity constraint in $Q(m)$ ensures that $s_m(X) = s_*(X)$ for all $X \in \text{dom}(s_*)$, satisfying the goal condition.

We now distinguish the different cases to show that A_t is $\forall$ -/ $\exists$ -/*relaxed* $\exists$ -step serializable in s_{t-1} based on the specific constraints of each encoding.

Case 1: To show: A_t is $\forall$ -step serializable.

Let $\langle a_1, \dots, a_k \rangle$ be an arbitrary sequence of actions such that $\{a_1, \dots, a_k\} = A_t$. We prove by induction that for all $1 \leq i \leq k$, the transition $o(\langle a_1, \dots, a_i \rangle, s_{t-1})$ is defined.

Base case: $i = 1$:

The integrity constraint checking preconditions (Line 1 of Listing 4) guarantees that a_1 is applicable in s_{t-1} , i.e., $o(a_1, s_{t-1})$ is defined.

Inductive step $i \mapsto i + 1$:

Assume $o(\langle a_1, \dots, a_i \rangle, s_{t-1})$ is defined. We must show that a_{i+1} is applicable in the resulting intermediate state $s_{temp} = o(\langle a_1, \dots, a_i \rangle, s_{t-1})$. Consider any fluent X in the precondition of a_{i+1} . If $X \notin \bigcup_{1 \leq j \leq i} \text{dom}(a_j^e)$, then no preceding action has modified X . Thus $s_{temp}(X) = s_{t-1}(X)$, and the precondition holds trivially because a_{i+1} is applicable

in s_{t-1} . Therefore, we focus on the critical case where $X \in \text{dom}(a_{i+1}^c) \cap \bigcup_{1 \leq j \leq i} \text{dom}(a_j^e)$. This implies there exists a preceding action a_j that has a postcondition on X . We distinguish two cases based on the effect of a_{i+1} itself:

(i) $X \notin \text{dom}(a_{i+1}^e)$:

The integrity constraint in Line 2 of Listing 4 requires that if a_{i+1} has a precondition on X but no postcondition, the value of X cannot change in the final state s_t . Since a_j writes X , we have $s_t(X) = a_j^e(X)$. The constraint implies $s_{t-1}(X) = s_t(X)$, meaning a_j effectively did not change the value relative to s_{t-1} . Thus $s_{temp}(X) = a_{i+1}^c(X)$.

(ii) $X \in \text{dom}(a_{i+1}^e)$:

The integrity constraint in Line 4 of Listing 4 ensures that if an action (a_{i+1}) has a precondition on X and sets a conflicting postcondition ($a^c(X) \neq a^e(X)$), it is marked as **single**. However, since another action a_j also acts on X (as established by the intersection), the integrity constraint in Line 5 would be violated. Since M is a stable model, this implies that **single**(X, t) $\notin M$ and $s_t(X) = a_{i+1}^e(X) = a_{i+1}^c(X) = s_{t-1}(X)$ must be the case.

In both cases, $s_{temp}(X) = a_{i+1}^c(X)$, so $o(\langle a_1, \dots, a_{i+1} \rangle, s_{t-1})$ is defined. Therefore, by induction, the entire sequence is valid, and A_t is $\forall$ -step serializable.

Case 2: To show: A_t is $\exists$ -step serializable.

The integrity constraint in Line 1 of Listing 5 guarantees that every action $a \in A_t$ is applicable in s_{t-1} , i.e., $o(a, s_{t-1})$ is defined. Furthermore, the integrity constraint in Line 6 ensures that **ready**(a, t) $\in M$ for each $a \in \mathcal{O}$ and the rules in Lines 4-5 that **apply**(a, t) $\in M$ for all $a \in A_t$. The rule defining **apply** (Lines 2-3) imposes a dependency: an action a can only be applied if all actions a' whose preconditions it invalidates are already **ready**. Since M is a stable model, there exists a derivation sequence of the **apply** atoms that respects these dependencies. Let $\langle a_1, \dots, a_k \rangle$ be the sequence of actions in A_t ordered by their derivation step. Consider any action a_i in the sequence and a fluent X in the precondition. Suppose that a preceding action a_j ($j < i$) invalidates this precondition, i.e., $a_j^e(X) \neq a_i^c(X)$. By the definition of **apply** in the Lines 2-3, a_j would have to wait for a_i to become **ready** before it could be applied. This would imply that a_i is derived before a_j , contradicting $j < i$. Therefore, for all $j < i$, a_j does not invalidate a_i . Thus, $s_{t-1}(X)$ is either preserved ($s_{t-1}(X) = s_t(X)$) or modified confluent ($a_j^e(X) = a_i^c(X)$), ensuring $o(\langle a_1, \dots, a_k \rangle, s_{t-1})$ is defined.

Regarding the alternative encoding using the **#edge** directive (Listing 6), the graph logic ensures serializability explicitly. For any $1 \leq t \leq m$, the directive defines the conflict graph $G_t = (A_t, E_t)$ where a directed edge $(a, a') \in E_t$ exists iff action a invalidates a precondition of action a' . Since $P^E(m)$ enforces that G_t is acyclic, there exists a sequence

$\langle a_1, \dots, a_k \rangle$ comprising the actions in A_t such that no edge connects any predecessor to a successor. Formally, for any a_i in the sequence, there is no a_j with $j < i$ such that $(a_j, a_i) \in E_t$. This absence of destructive edges from the past implies that for any X in the precondition, either no predecessor changes X or it is modified consistently, i.e. $s_{t-1}(X) = a_i^c(X)$ or $a_j^e(X) = a_i^c(X)$. Thus, $o(\langle a_1, \dots, a_i \rangle, s_{t-1})$ is defined for all i , and in particular $o(\langle a_1, \dots, a_k \rangle, s_{t-1})$ is defined, verifying that A_t is $\exists$ -step serializable.

Case 3: To show: A_t is *relaxed* $\exists$ -step serializable.

In view of the rule in Line 1 of Listing 7, a fluent value is reachable if $s_{t-1}(X) = V$, i.e., $\text{reach}(X, V, t) \in M$. The recursive structure of the rules for **apply** (Lines 3-4) and **reach** (Line 2) imposes a dependency ordering: An action a is only derived as **apply**(a,t) if all its preconditions are already in the extension of the **reach** predicate.

Since M is a stable model, there exists a finite derivation sequence for all atoms in M . Let $\langle a_1, \dots, a_k \rangle$ be the sequence of actions in A_t ordered by the step in which their corresponding **apply** atom was derived (or satisfied). We prove that this sequence is a valid serialization.

Consider any action a_i in this sequence and any fluent X in the precondition. The derivation of **apply**(a_i, t) requires that **reach**($X, a_i^c(X), t$) was previously derived. By the definition of the **reach** rules, this value must originate either from:

- (i) The state s_{t-1} (Line 1), which implies $s_{t-1}(X) = a_i^c(X)$, or
- (ii) The effect of a previously applied action a_j (Line 2), which implies $a_j \in \{a_1, \dots, a_{i-1}\}$ and $a_j^e(X) = a_i^c(X)$.

In both cases, the precondition of a_i is satisfied by the state constructed from the initial state s_{t-1} and the effects of the predecessors $\langle a_1, \dots, a_{i-1} \rangle$. Thus, $o(\langle a_1, \dots, a_k \rangle, s_{t-1})$ is defined.

□

3 Method

3.1 Pipeline Overview

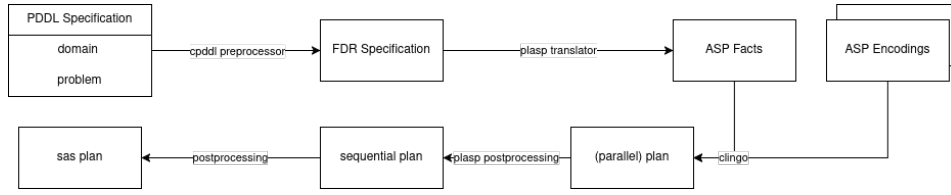


Figure 3.1: Pipeline for testing the different planning encodings

To systematically evaluate and compare the performance of the proposed parallel planning encodings against each other and the sequential baseline, we implemented an automated pipeline using the `lab` framework (Seipp et al. 2017). The benchmark¹ set consists of the unofficial collection of IPC instances provided by the Fast Downward project.

As illustrated in Figure 3.1, the workflow proceeds in four main stages:

1. **Preprocessing:** Starting with a planning task defined in PDDL (Planning Domain Definition Language), we use `cpddl` (Fišer 2025) to convert the problem into a Finite Domain Representation (FDR).
2. **Translation:** The `plasp` tool translates this FDR specification into a set of ASP facts. A detailed description of this translation process can be found in Dimopoulos, Gebser, et al. (2019).
3. **Solving:** In the core step, `clingo` combines these facts with our modified incremental encodings (sequential, $\forall$ -step, $\exists$ -step, or *relaxed* $\exists$ -step). The solver then searches for stable models corresponding to valid plans.
4. **Validation:** Finally, to ensure correctness, the resulting ASP-based plan (represented by the `occurs` atoms) is converted back into a sequential plan format. We employ the post-processing encoding provided by `plasp` for serialization and subsequently format the output to be readable by the plan validator VAL².

Each component of this pipeline involves specific configurations and optimizations, which will be detailed in the following sections.

¹<https://github.com/aibasael/downward-benchmarks>

²<https://github.com/KCL-Planning/VAL>

3.2 Encoding Adaptation

To integrate the theoretical encodings from [Dimopoulos, Gebser, et al. \(2019\)](#) presented in Section 2.2 into the automated pipeline, small changes are required to bridge the gap between the `plasp` output format and the internal predicates used by our logic programs. The necessary adaptations are shown in Listing 8.

```

1  #include <incmode>.
2  #program base.
3  fluent(X) :- variable(X).
4  fluent(X,V) :- initialState(X,V), fluent(X).
5  value(X,V) :- contains(X, V).
6  prec(A, X, V) :- precondition(A, X, V).
7  post(A, X, V) :- postcondition(A, _, X, V).
8  ...
9  :- fluent(X), #count{V : holds(X,V,0)} > 1.
10 ...
11 #program step(t).
12 ...
13 #show.
14 #show occurs(A, t): occurs(A, t).
```

Listing 8: Modifications to the core encoding

Lines 3-6 map the predicates generated by *plasp* (e.g., `initialState`, `contains`) to the predicates used in our encodings. Line 7 handles the action effects. By using the wildcard `_` for the second argument, we discard the specific effect identifiers. This simplification is chosen to reduce the complexity for the solver: explicitly tracking every effect could inflate the ground program and force *clingo* to manage a significantly larger set of atoms, which could degrade performance without necessarily improving the quality of the resulting plans. Therefore, it is assumed that the effect conditions are implicitly satisfied. Thus, the predicate `post` captures the potential effects of actions without encoding the conditions under which they occur. Line 9 performs a sanity check to ensure that the initial state is consistent (each fluent has at most one value). Finally, Lines 13-14 act as an output filter, ensuring that only the relevant `occurs` atoms are displayed in the final answer sets.

3.2.1 Post-Processing and Serialization

The stable models computed by *clingo* are output as sets of atoms (e.g., `occurs(action("move","a","b"), 1)`). Standard plan validators like *VAL*, however, require plans in a specific text-based format (often referred to as SAS plan or IPC format) structured as `time: (action args)`. To bridge this gap, we implemented a lightweight post-processing module in Python. Briefly, the script iterates through the output string to identify all `occurs` atoms using regular expressions. For each match, it extracts the time step t and the action arguments, stripping internal quotes and formatting delimiters. Finally, the extracted pairs are written to a file in a line-by-line format compatible with the validator. This conversion ensures that the parallel plans found by our encodings can be verified against the original PDDL domain and problem specification.

3.3 Optimizations

3.3.1 Mutexes

To further optimize the solving process, we utilize invariant constraints derived during the translation process. Specifically, we incorporate *mutex groups* (mutually exclusive sets of facts) provided by the *cpddl* preprocessor.

Definition 8 *mutex and mutex groups*. A set of facts $M \subseteq \mathcal{F}$ is called

- (i) a *mutex* $\iff$ for all reachable states $s : M \not\subseteq s$.
- (ii) a *mutex group* $\iff$ for all reachable states $s : |M \cap s| \leq 1$.

Intuitively, a mutex group represents a set of facts where no two facts can be true simultaneously. Enforcing these invariants as integrity constraints can significantly prune the search space by detecting invalid partial states early. We extend our encoding with the rules shown in Listing 9.


```

1  #program base.
2  ...
3  mutex(G,X) :- mutexGroup(G), contains(G,X,V), fluent(X,V).
4  mutex(G)    :- mutexGroup(G), #count{X : mutex(G,X)} > 1.
5  :- mutex(G), #count{X,V : holds(X,V,0), contains(G,X,V)} > 1.
6  ...
7  #program step(t).
8  ...
9  :- mutex(G), #count{X,V : holds(X,V,t), contains(G,X,V)} > 1.
10 ...

```

Listing 9: Expansion for mutexes

The mutex information is provided by *plasp* via the predicates `mutexGroup/1` and `contains/3`. In the base program, we first define which fluents belong to a mutex group (Line 3), where a group must contain at least two fluents (Line 4). The integrity constraint in Line 5 ensures that no more than one fluent of each mutex group holds in the initial state and the integrity constraint in Line 9 ensures the same for each state in timestep t .

3.3.2 Guess and Check Optimization

A crucial optimization for the $\exists$ -step encodings is the *Guess and Check* approach. The core idea is to decouple the search for action sets from the computationally expensive verification of acyclicity. Instead of enforcing serializability directly during the search, we first "guess" a candidate solution that satisfies all local constraints (preconditions, effects, confluence) but ignores potential cycles. In a second step, we "check" this candidate for circular invalidations.

Formally, we decompose the problem into a pair of logic programs $\langle G, C \rangle$, where G is the *generator* or *guesser* and C is the *checker*. A stable model M of G represents a valid plan if and only if the checker program, when grounded with the actions from M , is **unsatisfiable** (i.e., no cycle can be derived).

To implement the generator G , [Dimopoulos, Gebser, et al. \(2019\)](#) proposes using the core program $(S(t))$ from Listing 2 combined with the precondition checks (Line 1 of Listing 3, 4, or 5) and the problem instance facts. Consequently, a stable model M of G guarantees that:

1. Preconditions hold at $t - 1$ for all chosen actions.
2. Postconditions are applied and confluent (via Line 8 of Listing 2).

The only property remaining to be verified is the absence of circular invalidations (cycles in the conflict graph), where actions invalidate each other's preconditions (e.g., $a^e(X) \neq a'^c(X)$ and

$$a'^e(X') \neq a'^c(X').$$

This verification is performed by the program C , shown in Listing 10. The inputs to C are the action occurrences (`occurs/2`) from the candidate model M , along with the problem instance facts.

```

1  #include <incmode>.
2  #program check(t).
3  :- query(t), not cycle(t).
4  #program step(t).
5  prec(A, X, V, t) :- prec(A, X, V), occurs(A, t).
6  post(A, X, V, t) :- post(A, X, V), occurs(A, t).
7  action(A, t) :- occurs(A, t).
8  apply(A1, t) :- action(A1, t),
9      ready(A2, t): post(A1, X, V1, t), prec(A2, X, V2, t), A1 != A2 , V1 != V2.
10 ready(A, t) :- action(A, t), not occurs (A, t).
11 ready (A, t) :- apply(A, t).
12 cycle(t) :- action (A, t), not ready (A, t).

```

Listing 10: Encoding for detecting circular invalidation (Checker)

The checker works by attempting to construct a valid serialization order using the predicate `ready`:

- Like in the encoding of Listing 5, Lines 7-11 try to derive `ready(A, t)` for all actions that can be safely applied.
- Line 12 derives the atom `cycle(t)` if there exists any occurring action A that never becomes `ready`. This indicates that A is part of a invalidation cycle.
- Crucially, the program is designed to be **satisfiable** only if a cycle exists (Line 3).

Thus, if $C \cup M$ is satisfiable, a circular invalidation exists at some step t , meaning no valid permutation of the actions A_t exists. This implies the candidate is not serializable under the $\exists$ -step semantics and must be rejected. Conversely, if $C \cup M$ is unsatisfiable, no cycle was found, and the candidate M is a valid plan.

3.3.3 Heuristic

To further optimize the solving process, we employ domain-independent heuristics using the `#heuristic` directive of *clingo*. Following the approach described by Dimopoulos, Gebser, et al. (2019), we modify the solver’s search strategy by assigning specific attributes to atoms:

- *level* ($\in \mathbb{Z}$, default 0): Determines the decision priority. The solver selects atoms with higher levels before those with lower levels.
- *sign* ($\in \mathbb{Z}$, default 0): Determines the preferred truth value. A positive value (> 0) biases the solver to assign *true*, a negative value (< 0) to assign *false*. If the value is 0, the solver falls back to its default *sign* heuristic (e.g., VSIDS).

Listing 11 implements a *backward propagating* heuristic. The core idea is to leverage the goal condition and bias the search towards establishing the goal fluents as early as possible in the plan.

```

1 #program step(t).
2 #heuristic holds(X, V, t-1):    holds(X, V, t). [2147483647-t, true]
3 #heuristic holds(X, V, t-1):    not holds(X, V, t). [2147483647-t, false]
```

Listing 11: Backward propagating heuristic

The priority level is defined as $l = 2^{31} - 1 - t$ (the largest integer supported by *clingo*), assigning the highest priority to the earliest time steps (since smaller t yields a larger l). The logic in Lines 2 and 3 effectively projects state information backwards from t to $t - 1$:

- **Positive Bias** (Line 2): If *clingo* determines that $\text{holds}(X, V, t)$ is true, the heuristic activates and suggests setting $\text{holds}(X, V, t - 1)$ to true as well.
- **Negative Bias** (Line 3): Conversely, if $\text{holds}(X, V, t)$ becomes false, the heuristic biases $\text{holds}(X, V, t - 1)$ to be false.

Intuitively, this encourages the solver to satisfy the goal state s_* by assuming (unless forced otherwise by actions) that the desired fluents already held in the previous step $t - 1$. This creates a "pull" effect from the goal backwards to the initial state s_0 .

3.4 Experimental Setup

To evaluate the performance of the different planning encodings and the effectiveness of the proposed optimizations, we performed a systematic comparison across four distinct experimental configurations.

3.4.1 Configurations and Encodings

We tested the sequential encoding alongside the $\forall$ -step, both $\exists$ -step and *relaxed* $\exists$ -step encodings. Additionally, we evaluated the *Guess and Check* strategy with a fallback mechanism: if the Guess and Check approach fails to find a valid serializable plan (or implies a cycle), the system automatically falls back to a safe encoding (e.g. $\forall$ -step). In such cases, the reported runtime is

the sum of the time spent on the failed check and the time required by the fallback strategy. If this cumulative time exceeds the time limit, the instance is considered unsolved.

To isolate the impact of our optimizations, we ran the complete benchmark set in four configurations, varying the usage of mutexes and heuristics:

1. **Baseline:** No extra mutex detection, default solver heuristics.
2. **Mutexes (h2):** *cpddl* enabled with the h^2 option to compute additional mutex groups.
3. **Heuristics:** Using the backward propagating heuristic (Listing 11).
4. **Combined:** Both h^2 mutexes and backward heuristics enabled.

3.4.2 Benchmark Selection and Validation Criteria

Validation Criteria: For a planning task to be considered reliably *solved*, finding a candidate solution via *clingo* is necessary but not sufficient. We enforce a strict validation protocol: A run counts as solved if and only if:

1. The solver produces an answer set within the 30-minute time limit.
2. The extracted plan is successfully validated by the external validator *VAL*.

Any run where *clingo* reports a solution that *VAL* subsequently rejects is treated as an error and classified as unsolved.

Benchmark Domains: We evaluated our approach on a suite of benchmarks from the International Planning Competitions (IPC). The test set comprises domains used in previous competitions (e.g., IPC 1998-2018), specifically selected in their STRIPS formulations to ensure compatibility with our encodings. The selection covers a wide spectrum of planning complexity:

- *Classical Domains:* Standard benchmarks such as *Blocks*, *Logistics*, *Gripper*, and *Driverlog*.
- *Constraint-Heavy Domains:* Complex puzzles like *Sokoban*, *Freecell*, and *Parking*.
- *Recent Competitions:* Modern domains including *Agricola* (IPC 2018), *Snake* (IPC 2018), and *Quantum Layout* (IPC 2023).

This diverse set allows us to analyze the performance of parallel encodings across different problem structures (e.g., transportation, manipulation, construction).

3.4.3 Computational Environment

The experiments were executed on the *bwUniCluster 3.0* at the Karlsruhe Institute of Technology (KIT). We utilized compute nodes equipped with Intel Xeon Gold 6248R CPUs (3.0 GHz, 24 cores). To ensure comparability, each run was restricted to a single CPU core. We imposed a strict time limit of 30 minutes (1800 seconds) and a memory limit of 32 GiB for the execution of *clingo* and *cpddl* each. Runs exceeding these limits were aborted and recorded as unsolved (time-out).

4 Results

4.1 Overall Performance

To get an overview of the performance of the different encodings, we first analyze the aggregated results across all benchmark domains. In this section, we focus exclusively on the *baseline* configurations to establish a fundamental comparison. We compare the Sequential encoding (S) against the $\forall$ -step ($\forall$), the $\exists$ -step ($\exists$), the graph-based $\exists$ -step (E), the *relaxed* $\exists$ -step (R), and the Guess-and-Check strategy ($G\&C$).

Figure 4.1 illustrates the coverage (total number of solved instances) for each domain. For better readability, the data is split into two charts. Domains where no instances were solved by any algorithm are omitted. Figure 4.2 displays the average runtime. To ensure a fair comparison, this metric is calculated only on the intersection of instances that were successfully solved by *all* algorithms. Consequently, domains where no common set of solved instances exists are excluded from the runtime analysis.

4.1.1 Coverage Analysis

The coverage results in Figure 4.1 reveal several trends regarding the effectiveness of parallel planning.

Sequential vs. Parallel Performance: In domains that allow for a high degree of concurrency, parallel encodings significantly outperform the sequential approach. Notable examples include *airport*, *woodworking*, and both *logistics* variants. For instance, in *woodworking*, the sequential encoding (S) solves only 7 out of 50 instances, whereas all parallel encodings (with the exception of Guess-and-Check) solve the entire set. Similarly, in *floortile*, the sequential encoding fails to solve any instance (0/40), while the *relaxed* $\exists$ -step encoding achieves a coverage of 38/40. This confirms that the reduced time horizon T enabled by parallel semantics can effectively mitigate the search depth explosion inherent to sequential planning. This is on par with the experiment of [Dimopoulos, Gebser, et al. \(2019\)](#), where sequential encoding falls behind the parallel variants. However, in more complex or inherently sequential domains like *agricola* or *snake*, this performance advantage is not observed.

Comparison of Parallel Encodings: Comparing the different parallel semantics reveals a trade-off between *plan flexibility* (allowing shorter horizons) and *encoding complexity* (grounding

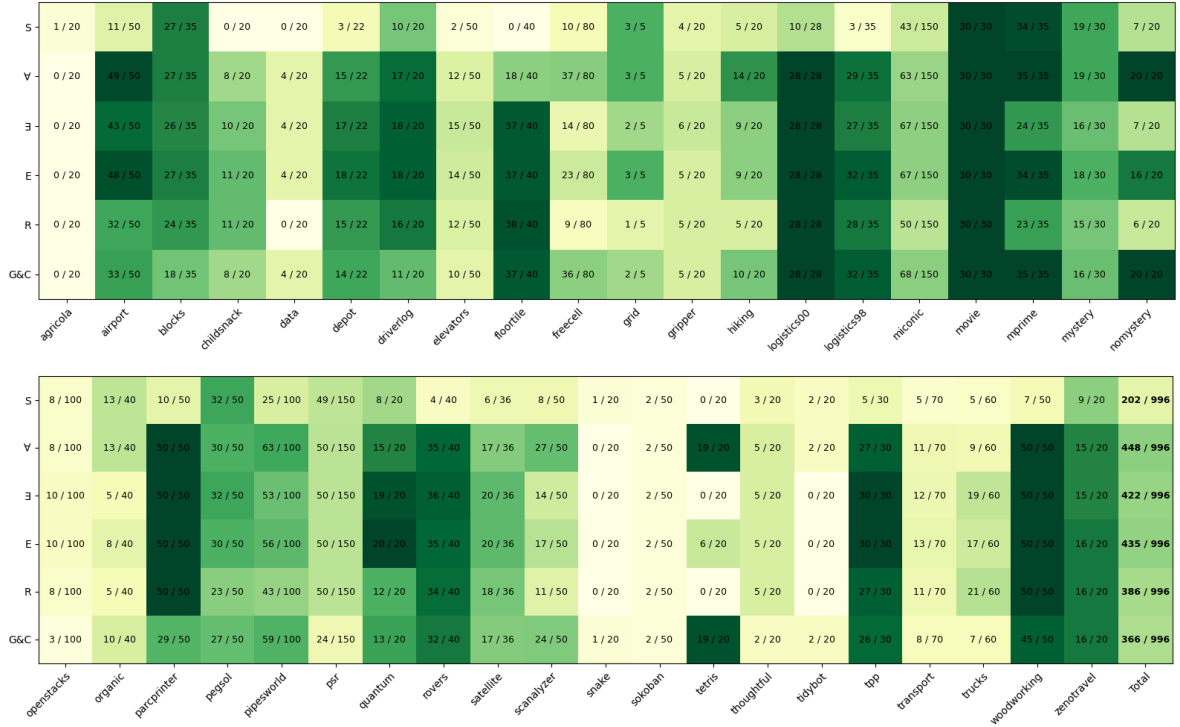


Figure 4.1: Coverage analysis: Number of solved instances per algorithm across benchmark domains. The heatmap indicates the absolute number of solved instances (left) out of the total available instances (right).

and solving overhead):

- **$\forall$ -step ($\forall$):** Although it enforces the strictest requirements for parallel execution, the $\forall$ -step encoding shows better performance in several domains (e.g., *tetris*, *parcprinter*), often outperforming even the more flexible variants in domains like *hiking* or *scanalyzer*. This suggests that for many IPC domains, the capability to interleave dependent actions is not critical. Consequently, the advantage of the $\forall$ -step semantics lies in its compactness: it requires fewer constraints and auxiliary atoms than the $\exists$ -step variants, thereby significantly reducing the burden on the grounder and solver. This is on par with the findings of [Dimopoulos, Gebser, et al. \(2019\)](#), where $\forall$ -step performs the best among the parallel encodings.
- **$\exists$ -step ($\exists$ / **E**):** The strength of the $\exists$ -step semantics is seen in domains with complex dependencies. A prime example is *floortile*, where the strict $\forall$ -step encoding solves only 18 out of 40 instances, whereas the $\exists$ -step encoding achieves a coverage of 37/40. This demonstrates that the increased flexibility of the $\exists$ -step semantics enables the discovery of plans in constrained environments where the conservative $\forall$ -step approach fails. Regarding the implementation variants, the graph-based alternative (*E*) generally performs on par with or marginally better than the standard constructive encoding ($\exists$) as in [Dimopoulos,](#)

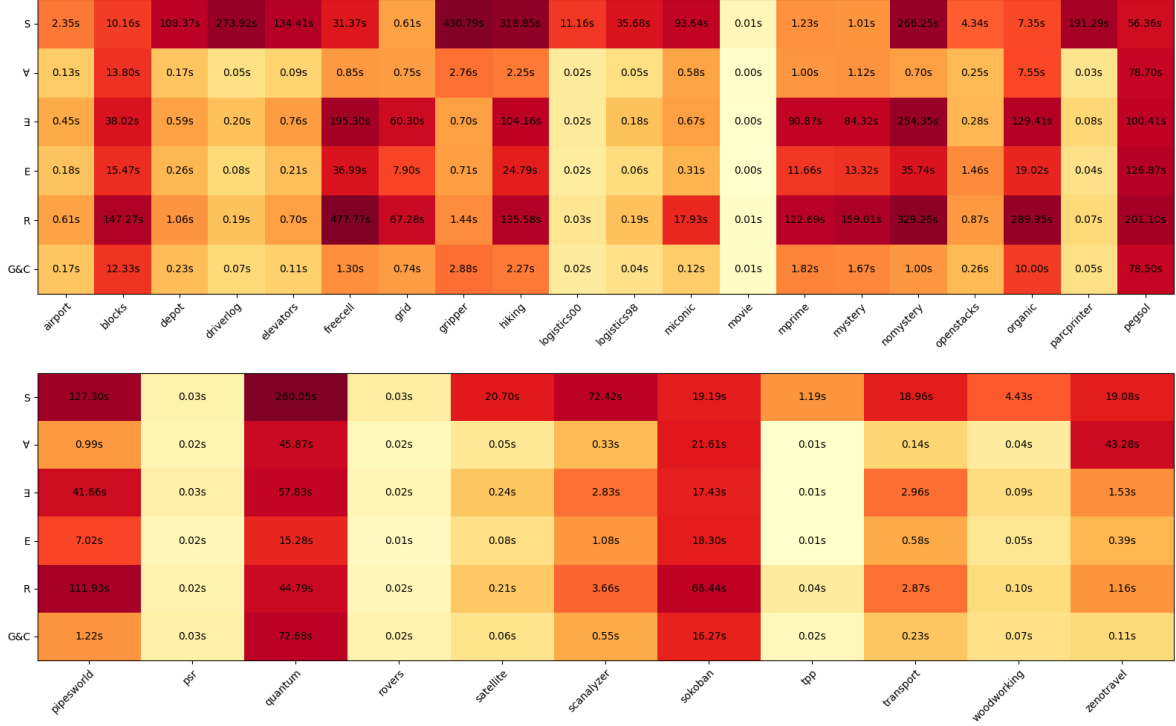


Figure 4.2: Runtime analysis: Average solving time (in seconds) for successfully solved instances. Red cells indicate higher runtimes, while light yellow cells indicate faster performance.

Gebser, et al. (2019), showing no significant deviation in total coverage.

- **Relaxed $\exists$ -step (R):** The relaxed encoding typically yields lower coverage compared to the standard $\exists$ -step variants, as observed in domains like *quantum-layout*. Although the *relaxed* $\exists$ -step encoding theoretically enables the shortest plan horizons by allowing actions to support each other within the same step, the computational cost is significant. The recursive definition of the reachability predicates (`reach/3`) probably increases the complexity of the ground program. Consequently, the overhead of solving these dense constraints often outweighs the benefits gained from a possible reduced time horizon.
- **Guess-and-Check (G&C):** The performance of the Guess-and-Check strategy is generally inferior to the direct encodings ($\exists$ and E). Crucially, when the validity check fails (i.e., the guessed plan contains a cycle), the pipeline must trigger the fallback mechanism. In these cases, the runtime accumulates (time for failed G&C + time for fallback), often leading to timeouts on instances that the pure $\forall$ -step or $\exists$ -step encodings could solve directly within the time limit. Consequently, this strategy proves less robust for general-purpose planning compared to the integrated approaches and even solves the fewest total instances compared to the other parallel encodings.

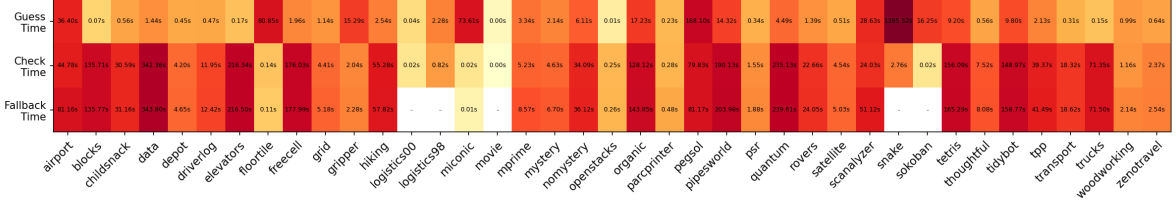


Figure 4.3: Average runtime for the Guess-and-Check strategy, broken down into the time spent on the initial guess and the subsequent check.

4.1.2 Runtime Analysis

Overview across Domains

Figure 4.2 provides an overview of the average runtimes across the evaluated domains. The sequential encoding (S) generally exhibits significantly higher runtimes compared to the parallel approaches. This discrepancy is particularly pronounced in domains with high concurrency potential, such as *Gripper*, where the average runtime for S is approximately 430 seconds, whereas the $\forall$ -step encoding requires less than 3 seconds. This confirms that the reduced plan horizon in parallel planning typically translates to reduced solving time.

Among the parallel variants, the $\forall$ -step encoding consistently achieves the lowest runtimes, rarely exceeding 100 seconds even on harder instances. This performance advantage can be attributed to the compactness of the encoding, which imposes the least burden on the grounder and solver.

The $\exists$ -step encodings ($\exists$ and E) display higher variability. While they allow for shorter plan lengths, the increased complexity of the compatibility checks often leads to longer solving times compared to $\forall$ -step, particularly in dependency-heavy domains like *Freecell*.

Detailed Pairwise Comparison

To investigate these performance characteristics and implementation differences in greater detail, Figure 4.4 provides pairwise scatter plots on common solved instances.

$\exists$ -step Implementation Variants (Fig. 4.4c): Comparing the two implementation variants reveals a distinct advantage for the graph-based approach. As illustrated in Figure 4.4c, the alternative $\exists$ -step encoding (E) outperforms the standard constructive encoding ($\exists$) in the majority of instances. Notably, in the *Nomystery* domain, E achieves a runtime reduction of over 200 seconds compared to $\exists$ on the largest instances, validating the efficiency of the `#edge` constraints.

$\forall$ -step vs. $\exists$ -step (Fig. 4.4b): The direct comparison between the two main parallel semantics highlights the trade-off mentioned above. Although the $\exists$ -step encoding is theoretically more

flexible, the scatter plot shows a cluster of points above the diagonal. This confirms that the $\forall$ -step encoding is frequently faster, suggesting that the overhead of grounding the complex $\exists$ -step compatibility constraints outweighs the benefits of a slightly reduced horizon.

$\forall$ -step vs. Guess-and-Check (Fig. 4.4d): Finally, the comparison with the Guess-and-Check strategy (*G&C*) challenges the assumption that the iterative approach necessarily incurs a high overhead. As shown in Figure 4.4d, the performance is largely on par with the $\forall$ -step encoding, with *G&C* frequently achieving even faster runtimes (points situated below the diagonal). This efficiency can be attributed to the reduced complexity of the *generator* component. Since the generator ignores circular dependency constraints, it produces a significantly smaller ground program than the full $\forall$ -step encoding. In domains where circular conflicts are rare, the first "guessed" plan is often valid. Consequently, the combined runtime of the lightweight generator and the subsequent check is often lower than solving the stricter, more complex constraints of the $\forall$ -step encoding directly.

4.2 Impact of Optimizations

To evaluate the effectiveness of the implemented optimizations, we compared the baseline encoding against configurations equipped with h^2 -mutexes, the backward-propagating heuristic, and a combination of both. Figure 4.5 provides a detailed runtime comparison of these configurations.

Heuristic Guidance (Fig. 4.5a): Contrary to expectations, the backward-propagating heuristic did not yield a significant improvement in coverage or runtime. As illustrated in Figure 4.5a, the data points align almost perfectly with the diagonal, indicating negligible deviation from the baseline performance. This suggests that the search from the goal backwards do not substantially reduce the search time compared to *clingo*'s internal heuristic.

Impact of more Mutexes (Fig. 4.5c & 4.5d): In contrast, the inclusion of the h^2 -algorithm for *cddl* resulted in a measurable performance gain. The impact is particularly pronounced in highly constrained domains such as *Agricola*. In this domain, the sequential encoding solves 7 instances (compared to only 1 in the baseline) and the $\forall$ -step encoding improves from 0 to 5 solved instances.

Analyzing the runtime reveals a distinct trade-off between solving power and preprocessing overhead:

- **Solver Efficiency (Fig. 4.5c):** When looking exclusively at the solver runtime (*clingo* time), the additional mutexes provide a drastic speedup. Most points lie well below the diagonal, showing that the mutex constraints effectively prune the search space.

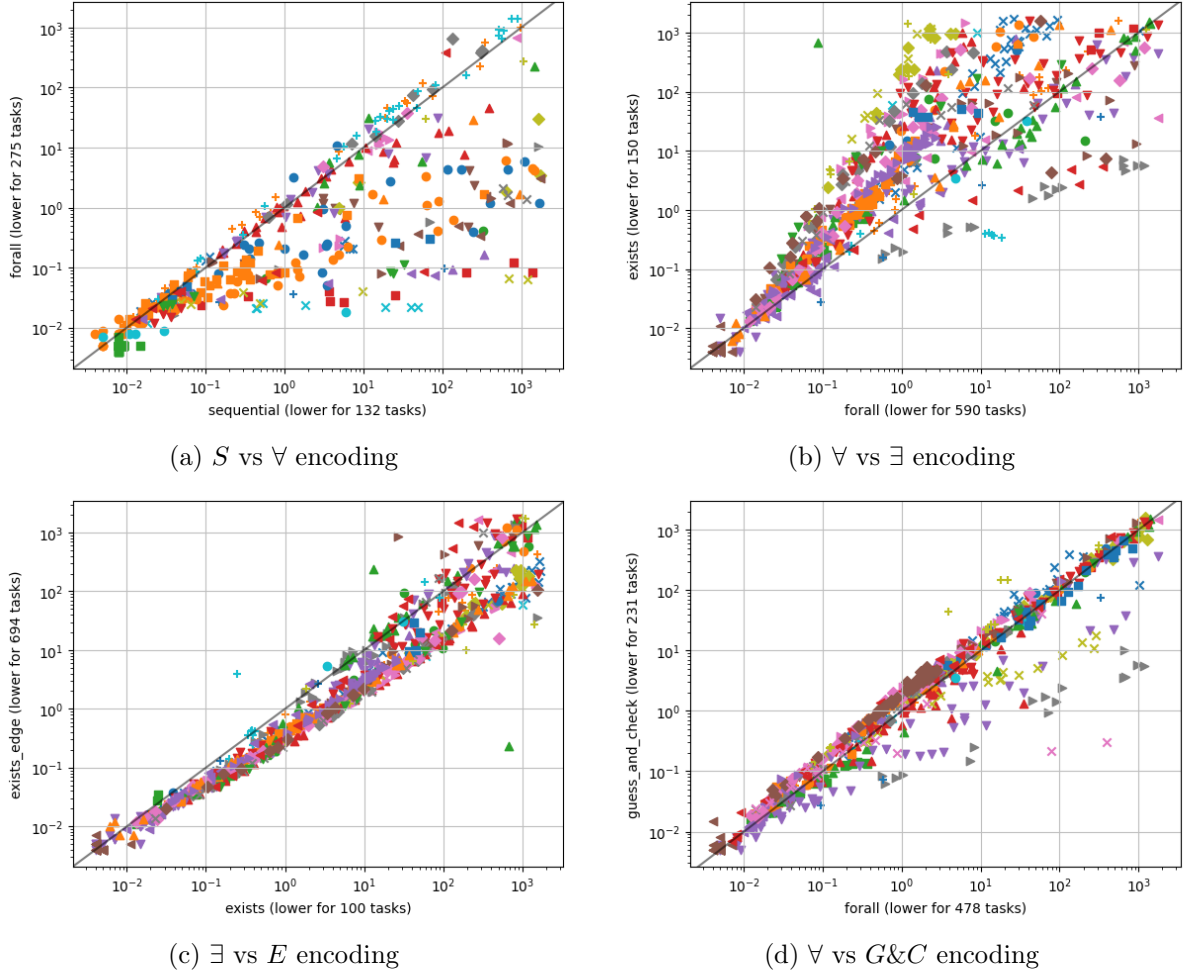


Figure 4.4: Runtime comparison between different encodings. Each point represents an instance solved by both encodings, with the x-coordinate indicating the runtime of the first encoding and the y-coordinate that of the second. Points below the diagonal indicate better performance of the first encoding.

- **Total Runtime (Fig. 4.5d):** When accounting for the full pipeline duration (including *cpddl* preprocessing), the picture changes. For many instances, the overall runtime does not significantly improve. In fact, for easier instances, the overhead of longer searching for mutexes outweighs the time saved during the plan search, leading to a longer total runtime compared to the baseline. This effect is particularly pronounced in the $\forall$ -step encoding, which is already compact and thus benefits less from additional pruning. In comparison, for the *relaxed* $\exists$ -step encoding, the total runtime with more mutexes is often lower than the baseline, indicating that "heavier" encodings profit more from the invariants.

Combined Configuration (Fig. 4.5b): As expected, adding the heuristic on top of the deeper mutex search does not yield further improvements. Figure 4.5b compares the encoding with

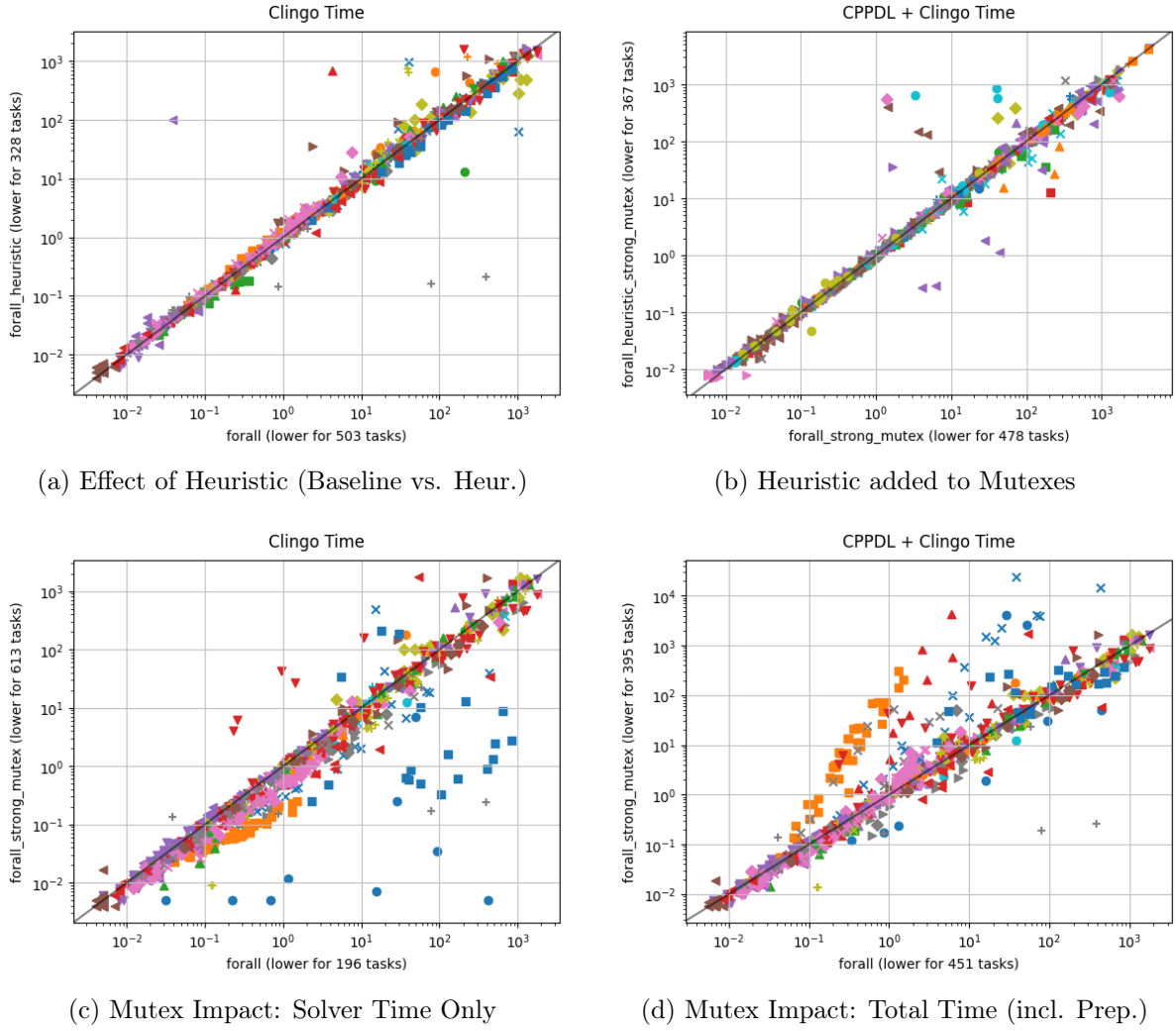


Figure 4.5: Runtime comparison of optimization techniques. Top row: The heuristic shows negligible impact (points on diagonal). Bottom row: Mutexes significantly reduce solver time (c), but preprocessing overhead affects total time on easy instances (d).

the mutex optimization against the combined version. The almost perfect alignment with the diagonal confirms that the heuristic remains ineffective even in the presence of more mutexes.

Figure 4.5b confirms that the system’s performance is driven almost exclusively by the constraints generated by the h^2 algorithm.

4.3 Conclusion and Future Work

4.3.1 Conclusion

The experimental evaluation leads to several key conclusions regarding the performance and characteristics of ASP-based planning encodings. First and foremost, the results demonstrate

that parallelism is the primary driver for efficiency in ASP planning. Parallel encodings significantly outperform the sequential baseline across almost all standard benchmarks, primarily because they drastically reduce the required planning horizon T . This reduction effectively mitigates the search depth explosion that renders sequential approaches infeasible for larger instances.

Regarding the comparison of parallel semantics, it became evident that compactness often outweighs theoretical flexibility. The strict $\forall$ -step encoding proved to be the most robust variant. Although the $\exists$ -step semantics theoretically allow for a higher compression of plans by merging more actions per step, the associated overhead of grounding the complex compatibility constraints often negates this advantage. However, in scenarios where the $\exists$ -step semantics is explicitly required to find a solution, the graph-based implementation (E) utilizing `#edge` constraints showed superior performance compared to the constructive approach.

Furthermore, the analysis highlights the critical importance of preprocessing. The integration of invariants, specifically mutex constraints derived from the h^2 algorithm, proved essential for solving highly constrained instances. In contrast, simple domain-independent heuristics, such as backward propagation, failed to provide a tangible benefit over the solver’s built-in strategies, suggesting that effective guidance in ASP requires more domain-specific control knowledge.

4.3.2 Future Work

While this work demonstrates the potential of ASP for STRIPS planning, several avenues for future research remain. One promising direction is the investigation of alternative strategies for determining the planning horizon. Instead of relying solely on *clingo*’s incremental mode, future work could explore algorithms such as fixed-horizon queries, as discussed by [Dimopoulos, Gebser, et al. \(2019\)](#), to potentially reduce the overhead of maintaining the solver state.

Additionally, the Guess-and-Check strategy could be further refined by implementing more adaptive fallbacks for for instance. Finally, as described in [Dimopoulos, Gebser, et al. \(2019\)](#) the encodings could be generalized to better support conditional effects and derived predicates or axiom rules.

Bibliography

- Gelfond, Michael and Vladimir Lifschitz (1988). “The Stable Model Semantics for Logic Programming”. In: *Proceedings of International Logic Programming Conference and Symposium*. Ed. by Robert Kowalski, Bowen, and Kenneth. MIT Press, pp. 1070–1080. URL: <http://www.cs.utexas.edu/users/ai-lab?gel88>.
- Kautz, Henry, D. A. McAllester, and Bart Selman (1996). “Encoding Plans in Propositional Logic”. In: Cambridge, Massachusetts, pp. 374–384.
- Dimopoulos, Yannis, Bernhard Nebel, and Jana Koehler (Oct. 1997). “Encoding Planning Problems in Nonmonotonic Logic Programs”. In: ISBN: 978-3-540-63912-1. DOI: [10.1007/3-540-63912-8_84](https://doi.org/10.1007/3-540-63912-8_84).
- Lifschitz, Vladimir (2002). “Answer set programming and plan generation”. In: *Artificial Intelligence* 138.1. Knowledge Representation and Logic Programming, pp. 39–54. ISSN: 0004-3702. DOI: [https://doi.org/10.1016/S0004-3702\(02\)00186-8](https://doi.org/10.1016/S0004-3702(02)00186-8). URL: <https://www.sciencedirect.com/science/article/pii/S0004370202001868>.
- Helmert, M. (2006). “The Fast Downward Planning System”. In: *Journal of Artificial Intelligence Research* 26, pp. 191–246.
- Rintanen, Jussi, Keijo Heljanko, and Ilkka Niemelä (2006). “Planning as satisfiability: parallel plans and algorithms for plan search”. In: *Artificial Intelligence* 170.12, pp. 1031–1080. ISSN: 0004-3702. DOI: [10.1016/j.artint.2006.08.002](https://doi.org/10.1016/j.artint.2006.08.002). URL: <https://www.sciencedirect.com/science/article/pii/S0004370206000774>.
- Wehrle, Martin and Jussi Rintanen (Dec. 2007). “Planning as Satisfiability with Relaxed $\exists$ -Step Plans.” In: vol. 4830, pp. 244–253. ISBN: 978-3-540-76926-2. DOI: [10.1007/978-3-540-76928-6_26](https://doi.org/10.1007/978-3-540-76928-6_26).
- Gebser, Martin, Roland Kaminski, Benjamin Kaufmann, and Torsten Schaub (May 2014). *Clingo = ASP + control: Preliminary report*. Tech. rep. 1405.3694. arXiv. URL: <https://arxiv.org/abs/1405.3694v1>.
- Bomanson, Jori et al. (Nov. 2016). “Answer Set Programming Modulo Acyclicity*”. In: *Fundamenta Informaticae* 147, pp. 63–91. DOI: [10.3233/FI-2016-1398](https://doi.org/10.3233/FI-2016-1398).
- Gebser, Martin, Roland Kaminski, Benjamin Kaufmann, Max Ostrowski, et al. (2016). “Theory Solving Made Easy with Clingo 5”. In: *Technical Communications of the 32nd International Conference on Logic Programming (ICLP 2016)*. Ed. by Manuel Carro et al. Vol. 52. Open Access Series in Informatics (OASICS). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-

- Zentrum für Informatik, 2:1–2:15. ISBN: 978-3-95977-007-1. DOI: [10.4230/OASICS.ICLP.2016.2](https://doi.org/10.4230/OASICS.ICLP.2016.2). URL: <https://drops.dagstuhl.de/entities/document/10.4230/OASICS.ICLP.2016.2>.
- Seipp, Jendrik et al. (2017). *Downward Lab*. <https://doi.org/10.5281/zenodo.790461>.
- Dimopoulos, Yannis, Martin Gebser, et al. (Jan. 2019). “plasp 3: Towards Effective ASP Planning”. In: *Theory and Practice of Logic Programming* 19.3, pp. 477–504. ISSN: 1475-3081. DOI: [10.1017/s1471068418000583](https://doi.org/10.1017/s1471068418000583). URL: <http://dx.doi.org/10.1017/S1471068418000583>.
- Gebser, Martin, Roland Kaminski, Benjamin Kaufmann, Marius Lindauer, et al. (2019). *Potassco User Guide*. <https://github.com/potassco/guide/releases/tag/v2.2.0>.
- Fišer, Daniel (Nov. 2025). *cpddl*. Version 1.5. DOI: [10.5281/zenodo.16423302](https://doi.org/10.5281/zenodo.16423302). URL: <https://doi.org/10.5281/zenodo.16423302>.